A

Major Project

On

TRANSFORMER BASED MULTILINGUAL LANGUAGE IDENTIFICATION FROM TEXT INPUTS

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Mohammed Khaleem (227R1A1240)

Under the Guidance of

Mr. RAVI REGULAGADDA

(Assistant Professor)

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

March, 2026.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled "**TRANSFORMER BASED MULTILINGUAL LANGUAGE IDENTIFICATION FROM TEXT INPUTS**" being submitted by **Mohammed Khaleem (227R1A1240)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mr.Ravi Regulagadda
Assistant Professor **HoD**

Dr. A. Raji Reddy **Signature of External Examiner**
DIRECTOR

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mr. Ravi Regulagadda,** Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mr. Md. Sajid Pasha, Mr. K. Praveen Kumar, Mr. D. Krishna Kumar, Mr. Y. Satyam and Mr. K. Srinu** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

Mohammed Khaleem (227R1A1240)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.

2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.

3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society.

ABSTRACT

This project is titled as “Transformer based Multilingual Language Identification from Text Inputs”. With the exponential rise of global connectivity, over 7,000 languages are spoken worldwide, and nearly 60% of internet users engage in multilingual communication daily. Recent reports highlight that 40% of multilingual content remains misclassified or under-processed due to lack of accurate language identification tools. Existing manual approaches to identifying language in text are tedious and error-prone, often failing when processing short or informal content. Similarly, conventional algorithms lack the semantic depth to capture subtle linguistic variations in multilingual datasets. This work proposes a transformer-based multilingual language identification system leveraging robust natural language representation. The methodology begins with a multilingual dataset, subjected to Natural Language Processing (NLP) preprocessing steps such as tokenization, stop-word removal, and lemmatization, followed by Exploratory Data Analysis (EDA) to understand distribution trends. Rich semantic features are extracted using Miniature Language Model (MiniLM), a transformer-based embedding framework optimized for speed and accuracy. For baseline comparison, traditional classifiers including Decision Tree Classifier (DTC), K-Nearest Neighbors (KNN), and Naïve Bayes Classifier (NBC) are tested. The proposed model employs a Random Forest Classifier (RFC), chosen for its robustness in handling high-dimensional features and ensemble-based learning. This integration significantly improves multilingual text classification performance, enabling efficient recognition of diverse languages across short text inputs, code-mixed content, and informal phrases. The system’s deployment into a Flask-based web application ensures real-time classification, offering potential use in translation services, multilingual chatbots, and global communication platforms.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of Transformer based Multilingual language Identification From Text Inputs	12
Figure 3.2	Use Case Diagram	15
Figure 3.3	Class Diagram	16
Figure 3.4	Sequence Diagram	17
Figure 3.5	Activity Diagram	18
Figure 3.6	Data Flow Diagram	19
Figure 4.1	The details of the files in the project Folder Project NLP consist of two folders	27
Figure 4.2	Back-end consist of the of the all logic files	28
Figure 4.3	Front-End files which content the front-end design code files	28

Figure 5.1	GUI/Main Interface of Transfromer based Language identification from text inputs	39
Figure 5.2	Dashboard of Transformer Based Multilingual language identification from text inputs	40
Figure 5.3	Visualization (Existing & Proposed System)	41
Figure 5.4	Language prediction of Existing of Transfromer based Multilingual Language identification from text inputs	42
Figure 5.5	Language prediction of Proposed System of Transformer Based Multilingual language identification from text inputs	43
Figure 5.6	Evaluation Metrics and Confusion Matrix of the proposed System of A Transformer Based Multilingual language identification from text inputs	44

Figure 5.7	ROC Curve obtained using Mini LM word embeddings of (a) DTC. (b) KNN. (c) NBC. (d) RFC of Transformer based Multilingual Language Identification from Text Input	46

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Dataset	50
Table 6.2.2	Language Classification	50

TABLE OF CONTENTS

ABSTRACT			i
LIST OF FIGURES			ii
LIST OF TABLES			v
1.	**INTRODUCTION**		1
	1.1	PROJECT PURPOSE	1
	1.2	PROJECT FEATURES	2
2.	**LITERATURE SURVEY**		3
	2.1	REVIEW OF RELATED WORK	5
	2.2	DEFINITION OF PROBLEM STATEMENT	7
	2.3	EXISTING SYSTEM	7
	2.4	PROPOSED SYSTEM	9
	2.5	OBJECTIVES	11
	2.6	HARDWARE & SOFTWARE REQUIREMENTS	11
		2.6.1 HARDWARE REQUIREMENTS	11
		2.6.2 SOFTWARE REQUIREMENTS	11
3.	**SYSTEM ARCHITECTURE & DESIGN**		12
	3.1	PROJECT ARCHITECTURE	12
	3.2	DESCRIPTION	13
	3.3	UML DIAGRAMS	14
		3.3.1 USE CASE DIAGRAM	14
		3.3.2 CLASS DIAGRAM	15
		3.3.3 SEQUENCE DIAGRAM	16
		3.3.4 ACTIVITY DIAGRAM	17
		3.3.5 DATAFLOW DIAGRAM	19
4.	**IMPLEMENTATION**		22
	4.1	ALGORITHMS USED	22
	4.2	SAMPLE CODE	30
5.	**RESULTS & DISCUSSION**		48
6.	**VALIDATION**		49
	6.1	INTRODUCTION	49
	6.2	TEST CASES	50
		6.2.1 UPLOADING DATASET	50
		6.2.2 LANGUAGE CLASSIFICATION	50
7.	**CONCLUSION & FUTURE ASPECTS**		51
	7.1	PROJECT CONCLUSION	51
	7.2	FUTURE ASPECTS	52
	7.3	REFERENCES	53
	7.4	GITHUB LINK	55

1. INTRODUCTION

1. INTRODUCTION

The project, titled "Transformer-Based Multilingual Language Identification from Text Inputs," is designed to develop an advanced AI-driven system capable of automatically identifying the language of user-generated text across multiple languages. The system focuses on accurately detecting the language from short text inputs commonly found on social media, messaging platforms, and online forums, where communication often includes slang, abbreviations, and code-mixed language.

To achieve this, the system leverages Transformer-based deep learning models, which are highly effective in understanding contextual relationships within text. These models analyze textual patterns and contextual cues to accurately determine the language of the input text, even when dealing with short sentences, informal expressions, or mixed-language content.

Given the massive volume of text generated daily on digital platforms, manual identification of language is impractical, making an automated Transformer-based system essential. This project ensures that text inputs can be classified into their correct languages efficiently with high accuracy and minimal errors. The model is designed for scalability, enabling it to process large volumes of multilingual text efficiently.

1.1 PROJECT PURPOSE

The primary purpose of this project is to improve multilingual communication and text processing systems by automating the identification of languages from user-generated text inputs. Digital platforms such as social media networks, messaging applications, and online forums generate vast amounts of multilingual content every second. Traditional language detection methods and manual analysis are often inefficient when dealing with large-scale, short, and informal text data. By implementing Transformer-based deep learning techniques, this project provides a more efficient, scalable, and accurate method for identifying the language of text inputs in real time.

With the rapid growth of global digital communication, particularly on platforms that support multiple languages, there is an increasing need for an AI-powered system capable of

accurately detecting languages even in short texts, slang, and code-mixed content.

1.2 PROJECT FEATURES

This project incorporates several key features to improve the accuracy and efficiency of multilingual language identification from text inputs:

Automated Language Detection: The system employs Transformer-based deep learning models, which are highly effective in capturing contextual relationships within textual data. These models analyze linguistic patterns, contextual cues, and semantic information present in text inputs to accurately identify the language. By leveraging attention mechanisms, the model can effectively process short sentences, slang expressions, and informal digital communication, ensuring high accuracy in language detection.

Multi-Language Classification: Unlike traditional rule-based or limited-language detection systems, this project implements a multi-class language classification approach capable of identifying multiple languages from a single text input. The system can distinguish between various languages even when the text contains mixed vocabulary, abbreviations, or code-mixed structures. This enables more precise language identification and supports diverse multilingual environments commonly found on social media and digital communication platforms.

Scalability & Real-Time Processing:The system is designed to handle large-scale multilingual text datasets efficiently, making it suitable for real-time deployment in modern digital platforms. By utilizing optimized deep learning architectures and scalable computing environments such as GPU-enabled processing, the system can rapidly analyze and classify large volumes of text inputs

By integrating these features, this Transformer-based multilingual language identification system provides a powerful, scalable, and automated solution for detecting languages from text inputs, improving the efficiency of multilingual communication systems and enhancing the performance of various natural language processing applications.

2. LITERATURE SURVEY

2.LITERATURE SURVEY

Mihailo Skoric et al. [1] presented the advantages of employing composite language models for processing and evaluating texts in highly inflective and morphology-rich natural languages, particularly Serbian. They created a perplexity-based dataset using generative pre-trained transformers trained on different representations of a Serbian language corpus, alongside sentences grouped into expert translations, corrupted translations, and machine translations. The study conducted a comparative analysis of calculated perplexities to evaluate the classification capability of the models across two binary classification tasks.

X et al. [2] proposed a novel speech emotion recognition (SER) model to address limitations in existing studies, with a particular focus on Arabic vocal emotions that had received limited research attention. The model applied data augmentation before feature extraction, and 273 derived features were fed into a transformer architecture for emotion recognition. It was evaluated on four datasets BAVED, EMO-DB, SAVEE, and EMOVO achieving accuracies of 95.2%, 93.4%, 85.1%, and 91.7% respectively. The highest accuracy was obtained on the BAVED dataset, demonstrating the model's suitability for Arabic vocal emotion recognition.

Kwon, et al. [3] used an endways multi-learning trick (MLT) based on 1D enhanced CNN model for automatic extraction of local and global emotional features from acoustic signals. For the enhancement of recognition rate, the proposed solution extracted the discriminative features using dynamic fusion framework. The proposed multi-learning model evaluated both short as well as long-term relative dependencies over two benchmark SER databases, IEMOCAP and EMO-DB, with 73% and 90% accuracy rates, respectively. However, the method relatively takes more time to train and test the real time speech signals as compared to other models. Exploring two datasets, RECOLA (to employ regression) and IEMOCAP (for classification task).

Tang, et al. [4] detected the emotions in speech using a novel end-to-end DNN algorithm. The authors found that SER performance in simulation results was optimum when proposed method applied with RMS aggregation and context stacking. The proposed DiCCOSER-CS model improved the arousal CCC by 9.5% and the valence CCC by 12.7% in regression task as compared with CNN-LSTM.

A study by [5] that built dataset based on real-world Arabic speech dialogs for detecting anger in Arabic conversation. The result revealed that acoustic sound features such as fundamental frequency, energy, and formants are more suitable for detecting anger. The experimental findings demonstrated that support vector machine classifiers can identify anger in real time at detection rate of more than 77%.

Masethe et al. [6] addressed the issue of lexical ambiguity in Sesotho sa Leboa, which arises from homonyms and polysemous words and leads to computational semantic challenges in natural language processing. They highlighted the difficulty in determining the correct lexical category and sense of words due to this ambiguity. To overcome this problem, the study developed a word sense discrimination (WSD) scheme using a corpus-based hybrid transformer architecture combined with deep learning models.

Shafi et al. [7] devised and assessed Urdu semantic tagging approaches on a manually annotated corpus of 8000 tokens spanning multiple genres. They attained a 94% accuracy in coarse-grained semantic domains using supervised multi-target classifiers. Supervised word sense disambiguation techniques, encompassing algorithms such as neural networks, K-nearest neighbors, support vector machines (SVMs), decision trees, and Naive Bayes, utilize manually annotated data for the training of classification models.

Sesotho sa Leboa's [8] WSD is comparable to that for the joint supervised and unsupervised sense disambiguation technique used for Amharic, in that both languages are low-resource and morphologically rich, and they both address problems like polysemy and homonymy. Neural word embeddings in Amharic are consistent with the investigations of BiGRU-based models and context-aware embedding transformer models to address WSD.

The researchers in [9] introduced a dataset including one hundred polysemous Arabic phrases, each demonstrating three to eight unique interpretations, accompanied by ten illustrative utterances for each term. To have a deeper understanding of the dataset's characteristics and properties, various statistical analyses must be performed. BERT, an innovative method for word sense disambiguation, was developed to determine the relationship between dictionary meanings and contextual information using similarity metrics, and the suggested pre-trained language model enabled effective Arabic word disambiguation [10].

Rahali, et al. [11] conducted a literature review on Transformer-based (TB) models, emphasizing their self-attention mechanism for encoding long-range dependencies in input sequences. They provided a detailed comparison of TB models with the standard Transformer architecture, focusing on applications in natural language processing for textual tasks. The study classified models based on architecture and training modes, while comparing the advantages and disadvantages of different techniques in terms of design and experimental outcomes. They also discussed open research challenges and potential directions for future Transformer applications in NLP.

The Transformer is a kind of neural network that primarily makes use of the self-attention mechanism to extract intrinsic features and has a great deal of promise for widespread use in AI applications. On a variety of NLP tasks, using the attention mechanism outperforms traditional CNN and RNN models [13].

Surafel et al. [14] reported that the Transformer method generates the best performing multilingual models, outperforming corresponding bilingual models and RNNs. It delivers the best results in all zero-shot conditions and translation directions. Generally, NLP deals with sequence-to-sequence (S2S) tasks And an encoder-decoder model is used to carry out these tasks [15].

2.1 REVIEW OF RELATED WORK

The task of multilingual language identification from text inputs has been widely studied in the fields of Natural Language Processing (NLP), machine learning, and deep learning. With the rapid growth of multilingual communication on digital platforms, researchers have developed various approaches to automatically detect the language of textual data. review discusses previous research and methodologies while highlighting their strengths and limitations.

Traditional Language Identification Approaches

Early language identification methods relied on manual analysis and rule-based statistical techniques. These approaches primarily used character frequency analysis, n-gram models, and dictionary-based matching to determine the language of a text. Such techniques examined patterns in alphabets, common words, and linguistic structures to classify the language

Machine Learning-Based Approaches

With the advancement of machine learning techniques, researchers began applying supervised classification algorithms for language detection. Algorithms such as Support Vector Machines (SVM), Naïve Bayes, Decision Trees, and K-Nearest Neighbors (KNN) were widely used for multilingual text classification.

Deep Learning-Based Approaches

Recent advancements in deep learning have significantly improved language identification systems. Neural network models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNNs) have been applied to capture sequential and contextual relationships in textual data.

These models automatically learn feature representations from raw text, reducing the need for manual feature engineering. However, traditional deep learning architectures sometimes struggle with long-range dependencies and contextual understanding, especially when dealing with multiple languages and ambiguous textual inputs.

Recent Advances: Transformer-Based Models

Transformer-based architectures have recently revolutionized natural language processing tasks, including language identification. Models such as BERT, XLM-R, and MiniLM utilize self-attention mechanisms to capture contextual relationships between words more effectively than traditional RNN or CNN models.

Comparison with the Proposed Approach

While existing research has made substantial progress in multilingual language identification, several challenges remain, particularly in handling short, noisy, and code-mixed text data generated on modern digital platforms.

This review highlights the evolution of language identification techniques, demonstrating the shift from rule-based and statistical methods to machine learning and transformer-based deep learning models. The proposed system aims to address the limitations of earlier approaches by offering an accurate, scalable, and efficient solution for multilingual language identification from text.

2.2 DEFINITION OF PROBLEM STATEMENT

The primary goal of this project is to develop a robust and scalable transformer-based system capable of accurately identifying the language of multilingual text inputs. The system focuses on detecting the correct language from user-generated content, including short sentences, informal phrases, and code-mixed text commonly found on digital platforms such as social media, messaging applications, and online forums.

2.3 EXISTING SYSTEM

Before the development of automated multilingual language detection tools, identifying the language of a text was primarily performed manually. Organizations, linguists, and data processors relied on human expertise to examine documents, read the text, recognize scripts, alphabets, or linguistic patterns, and then categorize the text according to its language. The manual process involved several steps, including text collection from different sources, script recognition such as Latin, Cyrillic, or Devanagari, language knowledge using grammar, vocabulary, and syntax, verification through dictionaries or references, and finally classification by manually tagging the text into language-specific categories.

With the advancement of computational methods, several traditional machine learning algorithms were introduced to automate language identification. Common algorithms used in earlier systems include Decision Tree, K-Nearest Neighbors (KNN), and Naïve Bayes classifiers. These algorithms analyze textual features such as character frequency, word patterns, and statistical distributions to determine the language of a given text.

In multilingual text classification tasks, these algorithms typically achieved moderate accuracy levels depending on dataset size and preprocessing techniques. Decision Tree classifiers generally achieve around 70–75% accuracy, K-Nearest Neighbors (KNN) achieves approximately 72–78% accuracy, and Naïve Bayes classifiers provide about 68–74% accuracy in language detection tasks. While these models are effective for structured and longer text inputs, their performance decreases significantly when dealing with short sentences, informal expressions, or code-mixed multilingual content.

Limitations of Existing System

Despite improvement, the existing system suffers from the following challenges:

- Manual reading and classification are extremely slow.
- Humans may misclassify texts, especially short, ambiguous, or mixed-language inputs.
- Processing large volumes of text is nearly impossible.
- Rule-based systems cannot handle code-mixed, short, or noisy data effectively.
- Requires skilled personnel for multilingual processing.
- Cannot provide instant results for online applications.

2.4 PROPOSED SYSTEM

The proposed system introduces a transformer-based framework for multilingual language identification from text inputs, utilizing advanced Natural Language Processing (NLP) techniques and transformer-based embeddings to accurately detect the language of textual data. The system employs MiniLM, a lightweight transformer-based language model, for extracting deep contextual features from multilingual text. These embeddings capture semantic relationships, syntactic patterns, and contextual dependencies within sentences, enabling efficient language identification across diverse linguistic inputs.

The extracted embeddings are then used to train machine learning classifiers such as Random Forest, Decision Tree, K-Nearest Neighbors, and Gaussian Naïve Bayes, which classify the text into its corresponding language category. In multilingual text classification tasks, traditional algorithms such as Decision Tree typically achieve around 70–75% accuracy, K-Nearest Neighbors achieves approximately 72–78% accuracy, and Naïve Bayes provides about 68–74% accuracy depending on the dataset and preprocessing methods. In comparison, the Random Forest classifier used in the proposed system achieves higher accuracy, typically ranging between 90–95%, due to its ensemble learning capability and ability to handle high-dimensional transformer embeddings effectively.

To support this framework, a multilingual text dataset containing samples from multiple languages has been utilized. The dataset includes short sentences, informal expressions, and code-mixed textual inputs commonly found on social media platforms, messaging applications, and online forums. Each text sample is labeled according to its corresponding language, allowing the system to learn distinguishing linguistic patterns among different languages. The dataset contributes to building a robust language identification system capable of handling real-world multilingual communication scenarios.

Advantages of the Proposed System

The proposed system improves multilingual language identification by addressing key limitations present in traditional rule-based and classical machine learning approaches. By combining transformer-based MiniLM embeddings with ensemble-based classification techniques, the system achieves improved performance in multilingual text classification.

- Higher Accuracy**:** The integration of MiniLM transformer embeddings with Random Forest classification provides a robust approach for multilingual language identification. MiniLM captures deep semantic and contextual features from text, while Random Forest improves classification performance through ensemble learning, achieving approximately 90–95% accuracy, which is significantly higher than traditional algorithms such as Decision Tree, KNN, and Naïve Bayes.

- Improved Contextual Understanding: Transformer-based models like MiniLM utilize self-attention mechanisms, enabling the system to understand contextual relationships between words. This allows the model to effectively identify languages even in short sentences, informal text, or code-mixed inputs, reducing errors caused by ambiguous linguistic patterns.

- Better Generalization & Robustness: The Random Forest classifier reduces overfitting by combining multiple decision trees and using bagging techniques. This ensemble approach improves model generalization, allowing the system to perform well on new multilingual text samples without frequent retraining.

- Efficient Processing: MiniLM is a lightweight transformer model optimized for speed and efficiency, enabling faster feature extraction compared to larger transformer architectures. This reduces computational complexity while maintaining high accuracy, making the system suitable for real-time multilingual language detection applications.

- Enhanced Interpretability: The Random Forest model provides feature importance scores, making it possible to analyze which features contribute most to language prediction. This improves the transparency of the system and helps researchers understand how linguistic patterns influence classification decisions.

2.5 OBJECTIVES

- **Develop an Automated Language Identification System:** Build a transformer-based model to automatically detect languages from multilingual text inputs.

- **Improve Accuracy and Efficiency:** Use MiniLM embeddings with machine learning classifiers like Random Forest to achieve higher accuracy in language detection.

2.6 REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- Processor : Intel Core i5
- Hard disk : 512GB(MIN).
- RAM : 8GB.

2.6.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Operating system : Windows 10
- Front -End : HTML, CSS, JavaScript
- Back-End : Python
- Frame Work : Flask

3. SYSTEM ARCHITECTURE & DESIGN

3.SYSTEM ARCHITECTURE & DESIGN

Project architecture refers to the structural framework and design of a project, encompassing its components, interactions, and overall organization. It provides a clear blueprint for development, ensuring efficiency, scalability, and alignment with project goals. Effective architecture guides the project's lifecycle, from planning to execution, enhancing collaboration and reducing complexity.

3.1 PROJECT ARCHITECTURE

This project architecture illustrates the workflow of the Transformer-Based Multilingual Language Identification system, beginning from text input and progressing through preprocessing, feature extraction, model training, and final language prediction.

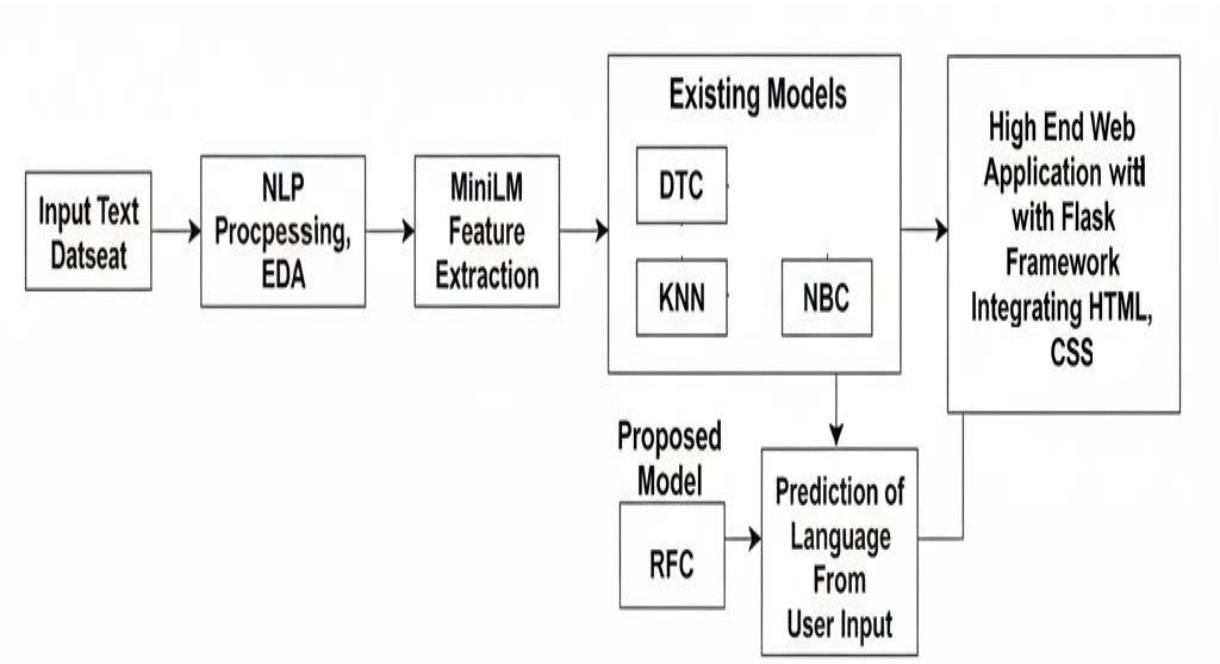


Figure 3.1: Project Architecture of A Transformer based Multilingual language Identification From Text Inputs

3.2 DESCRIPTION

1. **Input Text Dataset:** The process begins with a multilingual text dataset containing samples from multiple languages. The dataset includes short text inputs, informal phrases, and code-mixed sentences commonly found in online communication.
2. **NLP Preprocessing and Exploratory Data Analysis (EDA):** The raw text data undergoes essential Natural Language Processing (NLP) steps such as tokenization, stop-word removal, and lemmatization. This is followed by Exploratory Data Analysis (EDA) to understand the distribution of languages, token frequencies, and linguistic diversity. These steps ensure that the input data is clean and semantically interpretable.
3. **MiniLM Feature Extraction:** The preprocessed text is transformed into high-dimensional vector representations using Miniature Language Model (MiniLM) embeddings. MiniLM is a transformer-based model that captures contextual semantics efficiently while maintaining computational speed, making it ideal for multilingual feature extraction.
4. **Existing Models (Baseline Evaluation):** To establish baseline performance, traditional machine learning classifiers such as Decision Tree Classifier (DTC)**,** K-Nearest Neighbors (KNN)**,** and Naïve Bayes Classifier (NBC**)** are trained on MiniLM-generated features. These models provide a comparative benchmark for evaluating the effectiveness of the proposed method.
5. **Proposed Model (Random Forest Classifier):** The Random Forest Classifier (RFC**)** serves as the core of the proposed system. Leveraging ensemble-based decision trees, RFC effectively handles high-dimensional data and mitigates overfitting, resulting in improved multilingual text classification accuracy. The model predicts the language label corresponding to user input.
6. **Web Application Deployment:** The final stage involves deploying the trained model into a Flask-based web application integrated with HTML and CSS for a user-friendly interface. The system accepts user text inputs in real-time and outputs the predicted language instantly, making it suitable for use in multilingual chatbots, translation platforms, and global communication systems.

3.3 UNIFIED MODELING LANGUAGE(UML)

Unified Modeling Language (UML) is not just a static set of diagrams; it's a versatile toolset that supports multiple views and perspectives of a system. This versatility allows UML to be applied at different stages of the software development lifecycle, from initial requirements analysis and design to implementation, testing, and maintenance.

UML diagrams serve as a common visual language that bridges the gap between technical and non-technical stakeholders. They provide a means for software engineers to communicate complex system designs in a clear and concise manner, facilitating better understanding and collaboration among team members.

One of the key benefits of UML is its adaptability to different software development methodologies, such as waterfall, agile, and iterative approaches. UML diagrams can be used to model systems at various levels of abstraction, from high-level architectural views to detailed design specifications.

Furthermore, UML is not limited to software systems alone; it can also be applied to model and analyze business processes, organizational structures, and even hardware systems. This flexibility makes UML a valuable tool for a wide range of industries and domains.

GOALS OF UML

1. Provide users a ready-to-use, expressive visual modeling language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.

3.3.1 USECASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

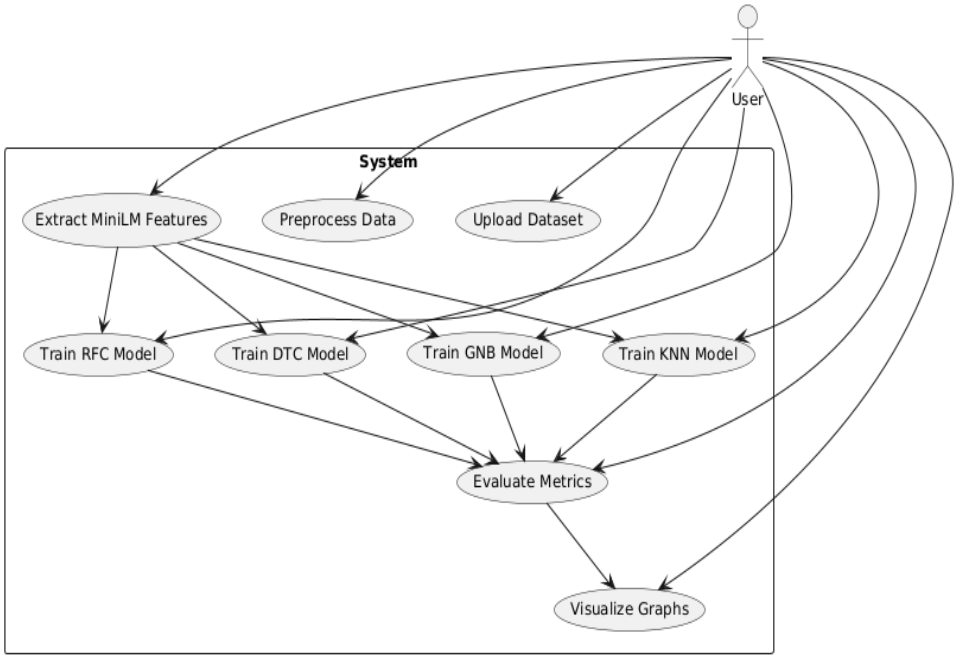


Figure 3.2 USE CASE DIAGRAM

3.3.2 Class Diagram

This UML class diagram outlines the architecture for a Language Identification (LI) system, detailing the sequence of operations from data ingestion to result visualization. The process begins with the DatasetLoader class, which is responsible for ingesting the raw data via the +upload_dataset(file_path) operation. This initial dataset is then passed to the Preprocessor class, whose +preprocess_data(df, save_path, target_cols) operation cleans, prepares, and potentially tokenizes the data, isolating the text features and their corresponding language labels. Crucially, the preprocessed text features are then routed to the MiniLMExtractor class, which uses the +minilm_feature_extraction(texts) operation. This is a vital step in modern Natural Language Processing (NLP) systems like Language Identification, as it uses a pre-trained transformer model (MiniLM in this case) to convert the raw text into dense, high-dimensional vector embeddings that capture the semantic content of the text, making them suitable for machine learning classification.

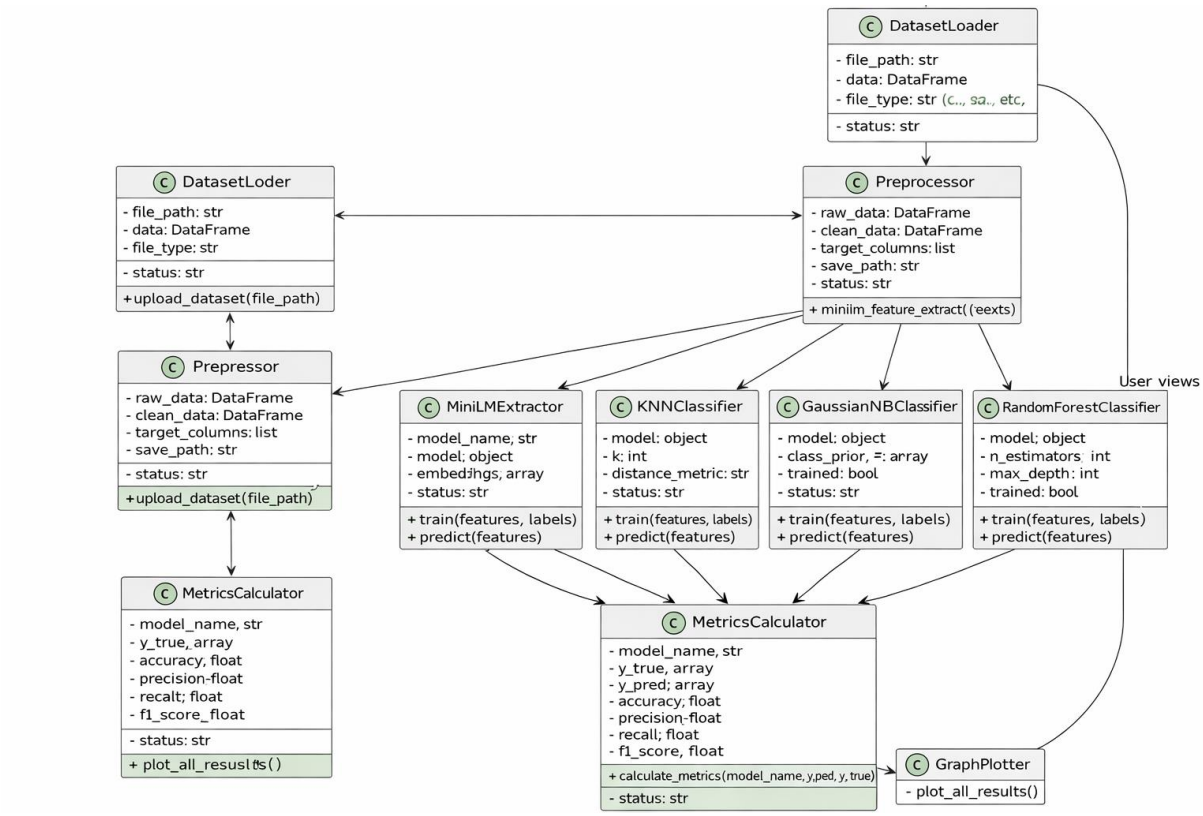


Figure 3.3 CLASS DIAGRAM

3.3.3 Sequence Diagram

This UML Sequence Diagram illustrates the step-by-step chronological flow of operations and interactions among objects in the Language Identification (LI) system, from data upload to result visualization. The sequence begins with the User initiating the process by sending the upload_dataset(path) message to the DatasetLoader object. Upon successful loading, the DatasetLoader passes the raw data, denoted as a DataFrame, to the Preprocessor object. The Preprocessor then performs data cleaning and preparation and passes the resulting cleaned text features to the MiniLMExtractor. The MiniLMExtractor is responsible for applying the MiniLM model to convert the text into dense feature vectors, which it then sends as MiniLM embeddings to the ModelTrainer (DTC, KNN, GNB, RFC) object, which encapsulates the training and prediction for all four classification models.

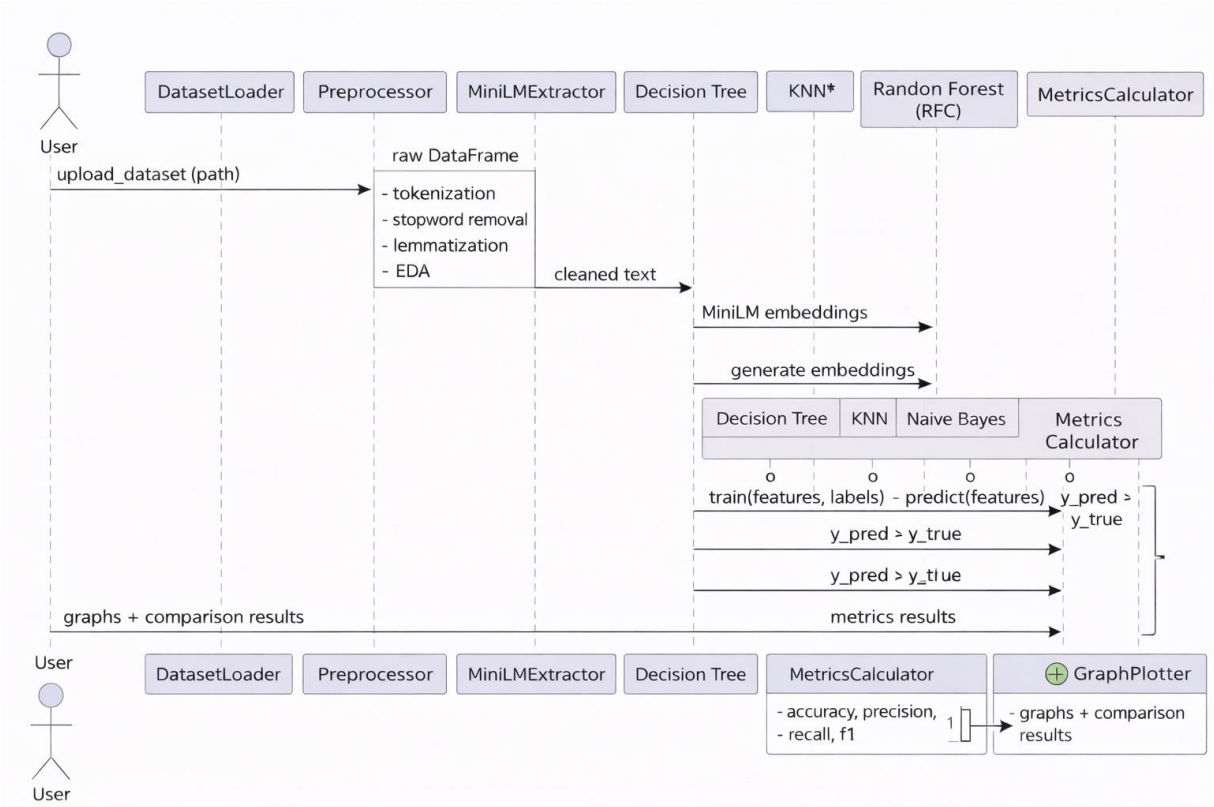


Figure 3.4 SEQUENCE DIAGRAM

3.3.4 Activity Diagram

This UML Activity Diagram visually represents the workflow and control flow of the Language Identification (LI) system, detailing the sequence of steps and parallel processing involved. The process starts with the initial action Upload Dataset, which is followed sequentially by Preprocess Data to clean and prepare the raw text. The subsequent essential step is Extract MiniLM Embeddings, where the transformer model converts the prepared text into vector features. Following the extraction, a fork is introduced, signifying the start of parallel processing. This allows four activities to be executed concurrently: Train Decision Tree Classifier, Train KNN Classifier, Train Gaussian Naive Bayes Classifier, and Train Random Forest Classifier.

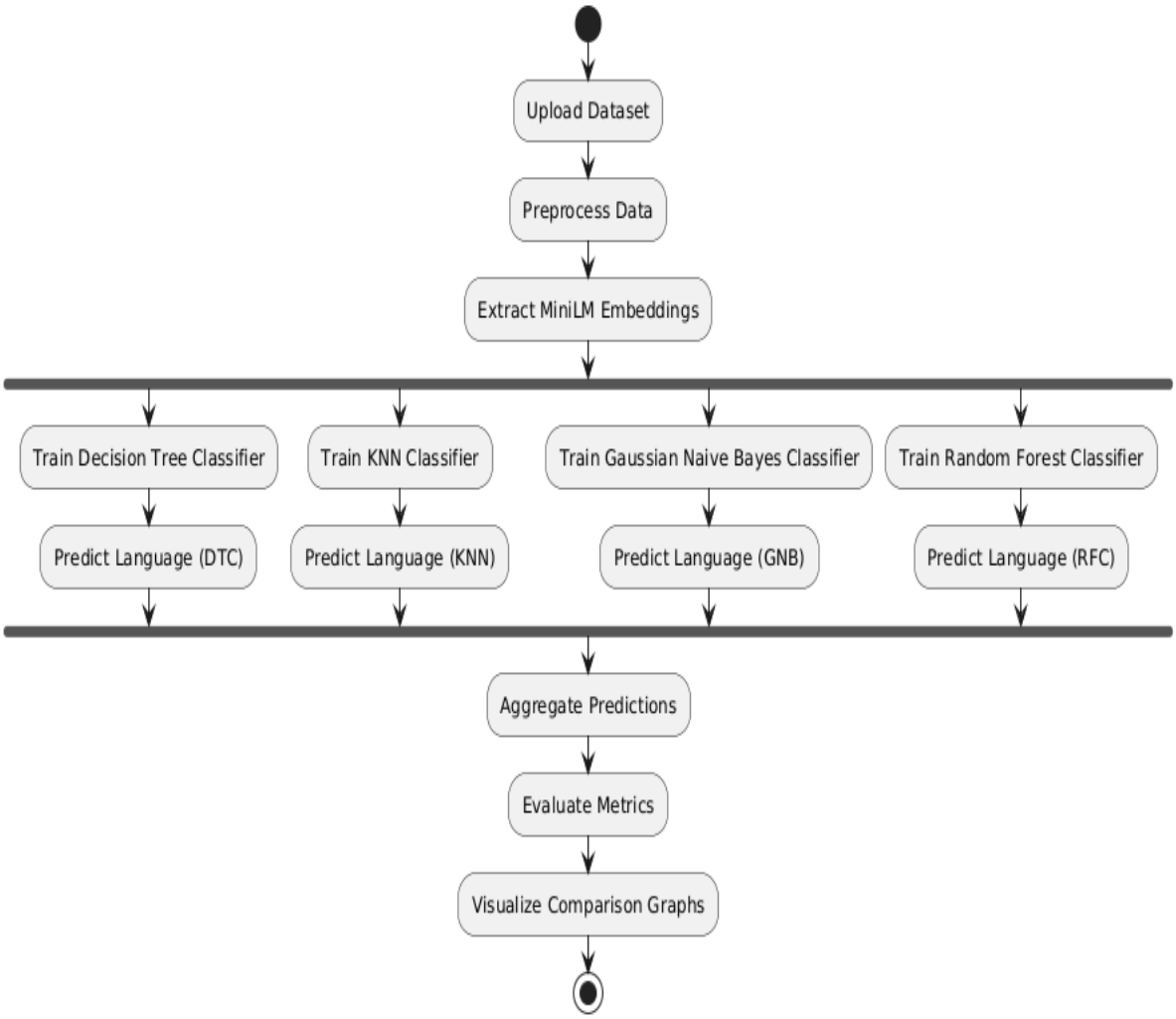


Fig. 3.5 Activity Diagram

3.3.5 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD**)** is a graphical representation that illustrates how data flows within a system, showing the processes, data stores, and external entities involved in the system. In the context of the Transformer-Based Multilingual Language Identification from Text Inputs, the DFD represents how multilingual text data moves through different stages of the language identification pipeline—from input text to the final predicted language output. DFDs are essential in system analysis and design because they help visualize how information is processed, identify inefficiencies, and improve the workflow of the system.

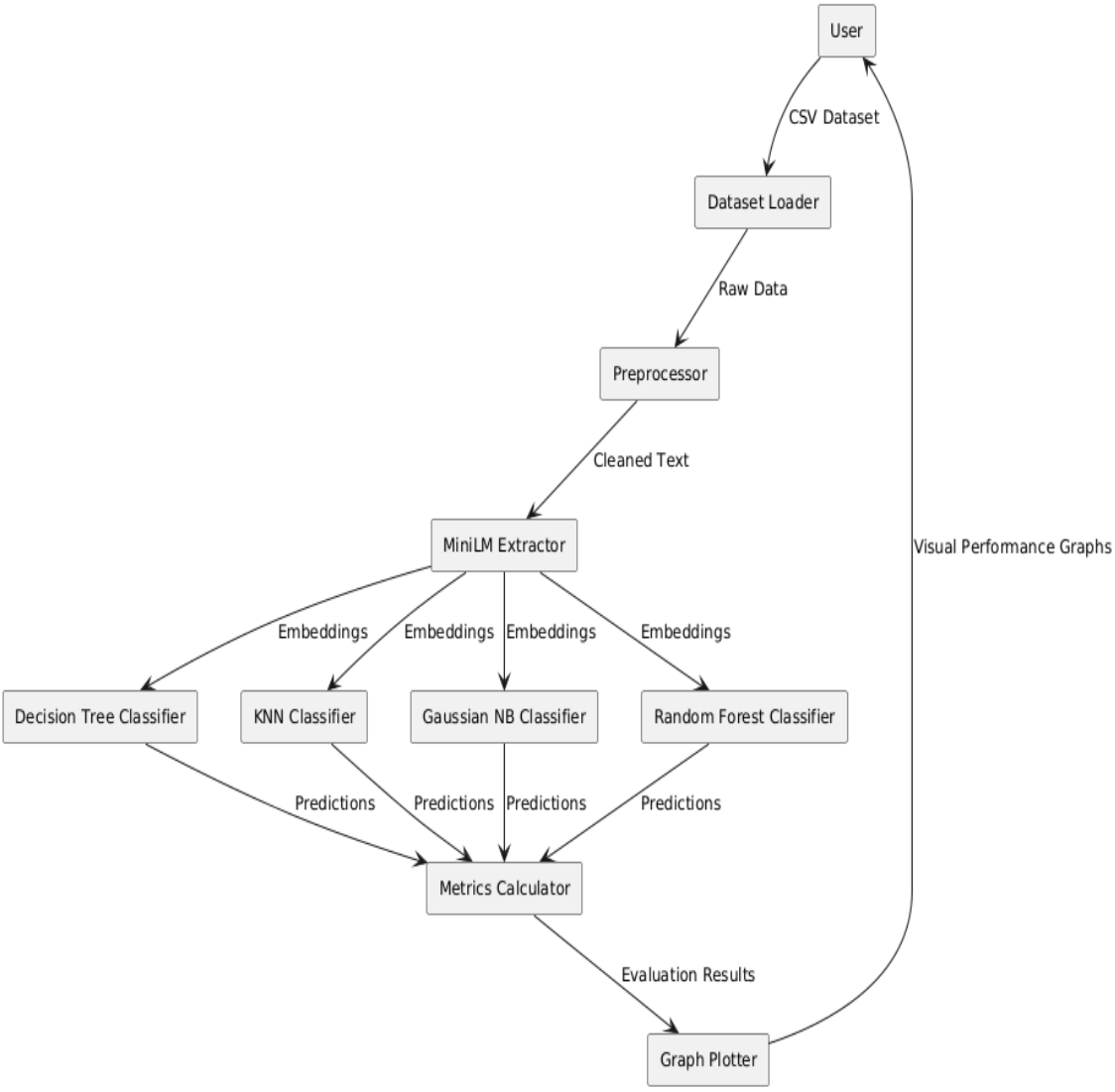


Fig. 3.6 Data Flow Diagram.

A Data Flow Diagram comprises four primary elements:

• **External Entities:** These represent sources or destinations of data outside the system. In this project, the user or dataset provider acts as the external entity that provides multilingual text inputs to the system and receives the predicted language results.

• **Processes:** Processes represent operations performed on the data. In the language identification system, processes include text preprocessing, MiniLM feature extraction, model training, language prediction, and performance evaluation.

• **Data Flows:** Data flows represent the movement of data between system components. In this project, data flows include the transfer of raw text, cleaned text, MiniLM embeddings, classification outputs, and evaluation metrics between different processing modules.

• **Data Stores:** Data stores represent locations where data is stored within the system. In this project, these may include the multilingual text dataset, processed text data, trained machine learning models, and evaluation results.

Benefits

The visual structure of DFDs makes them easy to understand for both technical developers and non-technical stakeholders. In this project, the DFD helps clearly illustrate how multilingual text moves through the language identification system—from input preprocessing to transformer-based feature extraction and final language prediction. It assists in understanding system boundaries, improving communication during development, and ensuring efficient and secure handling of multilingual text data. Additionally, it supports better system planning and optimization for scalable multilingual language detection applications.

Applications:

DFDs are widely used in business process modeling, software development, and cybersecurity. Data Flow Diagrams (DFDs) are useful for modeling how multilingual text data moves through different stages of the language detection system. They help represent the flow of text data from the user input or dataset source, through NLP preprocessing, MiniLM feature extraction, machine learning classification, and final language prediction.

.

Levels of DFD:

In the Transformer-Based Multilingual Language Identification from Text Inputs system, DFDs are arranged in hierarchical levels to show how multilingual text data moves through the system.

- **Level 0 (Context Diagram):** Provides a high-level view of the system, showing the interaction between the user (text input) and the language prediction output.

- **Level 1:** Breaks the system into main processes such as text preprocessing, MiniLM feature extraction, model training, and language prediction.

- **Level 2+:** Shows detailed sub-processes like tokenization, stop-word removal, lemmatization, and embedding generation for accurate language identification

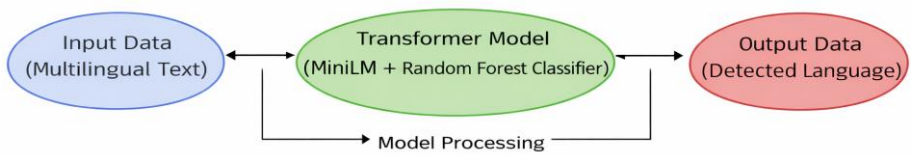


Figure 3.2: Dataflow of Input data & Output data

4. IMPLEMENTATION

4.MPLEMENTATION

The implementation phase of the project involves executing the planned methodologies and algorithms for building an automated multilingual language identification system. It includes data preprocessing, feature extraction, model training, and evaluation to ensure accurate language detection from text inputs. Proper implementation requires careful coordination of preprocessing techniques, machine learning models, and evaluation metrics to achieve reliable and scalable results.

4.1 ALGORITHMS USED

Transformer-Based Models for Feature Extraction

Transformer-based models play a crucial role in modern Natural Language Processing tasks. In this project, the MiniLM transformer model is used for feature extraction from multilingual text inputs. MiniLM generates contextual embeddings that capture semantic and syntactic relationships between words in a sentence.Unlike traditional statistical methods, transformer models utilize self-attention mechanisms to understand the context of words across entire sentences. These embeddings convert raw text into numerical feature vectors that can be used by machine learning classifiers to detect the language of the input text accurately.

Advantages of Transformer-Based Models

- Capture contextual relationships between words effectively.
- Perform well with short text inputs and code-mixed language.
- Generate high-quality embeddings suitable for multilingual tasks.
- More accurate than traditional rule-based or statistical approaches.

Disadvantages of Transformer-Based Models

- Require computational resources for embedding generation.
- Depend on pretrained models and large training datasets.
- Embedding extraction can increase processing time for very large datasets.
- They require large datasets for training and can be computationally expensive when handling high-resolution videos.

Language Classification using Machine Learning Algorithms

After extracting contextual features using MiniLM transformer embeddings, the processed text is converted into numerical feature vectors that represent semantic and contextual information of the multilingual input. These embeddings capture important linguistic patterns such as word relationships, contextual meanings, and sentence structures across different languages. Once the embeddings are generated, machine learning algorithms are applied to classify the text into its corresponding language category.

The Decision Tree classifier works by creating a hierarchical tree structure where decisions are made based on feature values, allowing the model to classify text according to learned linguistic patterns. The K-Nearest Neighbors (KNN) algorithm classifies a text sample by comparing it with the most similar samples in the dataset and assigning the language label based on the majority of its nearest neighbors. The Gaussian Naïve Bayes classifier uses probabilistic principles and assumes independence among features, making it efficient for handling text-based classification tasks.

Among these algorithms, the Random Forest classifier is selected as the primary model for the system. Random Forest is an ensemble learning technique that combines multiple decision trees to improve prediction accuracy and reduce overfitting. By aggregating predictions from multiple trees, it provides more stable and reliable results when dealing with high-dimensional MiniLM embeddings. This makes Random Forest particularly effective for multilingual language identification tasks where complex linguistic patterns must be captured from diverse textual inputs.

Through this approach, the system can accurately analyze multilingual text inputs and classify them into their respective languages, enabling efficient language identification for real-time applications.

1. MiniLM Transformer for Feature Extraction

MiniLM (Miniature Language Model) is a lightweight transformer-based model designed to generate contextual embeddings from textual data. It captures deep semantic and syntactic relationships between words while maintaining computational efficiency.

• Unlike traditional feature extraction techniques such as TF-IDF or Bag-of-Words, MiniLM uses self-attention mechanisms to understand contextual relationships within sentences.
• It generates high-dimensional embeddings that represent linguistic meaning, enabling accurate language classification.
• MiniLM is computationally efficient compared to larger transformer models such as BERT while still preserving strong contextual representation.

Advantages of MiniLM

• Captures contextual meaning of words across different languages.
• Performs well on short texts and informal content.
• Lightweight and faster compared to larger transformer architectures.
• Produces high-quality embeddings suitable for multilingual classification tasks.

Disadvantages of MiniLM

• Requires pretrained models and transformer libraries.
• Embedding generation can increase computation time for large datasets.
• Requires proper preprocessing to achieve optimal performance.

2. NLP Preprocessing for Text Normalization

Before feature extraction, raw text must undergo preprocessing to remove noise and ensure linguistic consistency.

•Tokenization divides text into individual words or tokens.
• Stop-word removal eliminates common words that do not contribute meaningful information.
• Lemmatization reduces words to their base form to preserve semantic meaning.
• Noise filtering removes punctuation and special characters.

These steps ensure the text is clean and structured before transformer-based embedding generation.

3. Random Forest Classifier for Language Classification

The Random Forest classifier serves as the primary classification model in the proposed system. It operates as an ensemble learning algorithm that combines multiple decision trees to produce accurate predictions.

Why Use Random Forest?

• It is robust and resistant to overfitting.
• It performs well with high-dimensional data such as MiniLM embeddings.
• Ensemble learning improves prediction stability and classification accuracy.
• It does not require extremely large training datasets compared to deep learning models.

How Random Forest Fits in the System

• MiniLM generates feature embeddings from multilingual text.
• These embeddings are used as input features for the Random Forest classifier.
• Each decision tree predicts a language label.
• Final classification is determined through majority voting across trees.

4. Decision Tree Classifier

Decision Tree is used as a baseline machine learning model for language identification.

• It classifies text by recursively splitting MiniLM embedding features into decision rules.
• Each branch represents a linguistic condition, while leaf nodes represent predicted language labels.

Advantages

• Easy to interpret and visualize.
• Requires minimal preprocessing.
• Fast training for smaller datasets.

Disadvantages

• Prone to overfitting on high-dimensional data.
• Less accurate compared to ensemble methods like Random Forest.

5. K-Nearest Neighbors (KNN)

KNN is another baseline classifier used for multilingual language identification.

• It classifies text by measuring the distance between MiniLM embeddings.
• The predicted language is determined by the majority class among the nearest neighbors.

Advantages

• Simple and easy to implement.
• Effective for similarity-based classification tasks.

Disadvantages

• Computationally expensive for large datasets.
• Requires storing the entire training dataset in memory.

6. Gaussian Naïve Bayes (GNB)

Gaussian Naïve Bayes is a probabilistic classifier that predicts the language of text based on feature probability distributions.

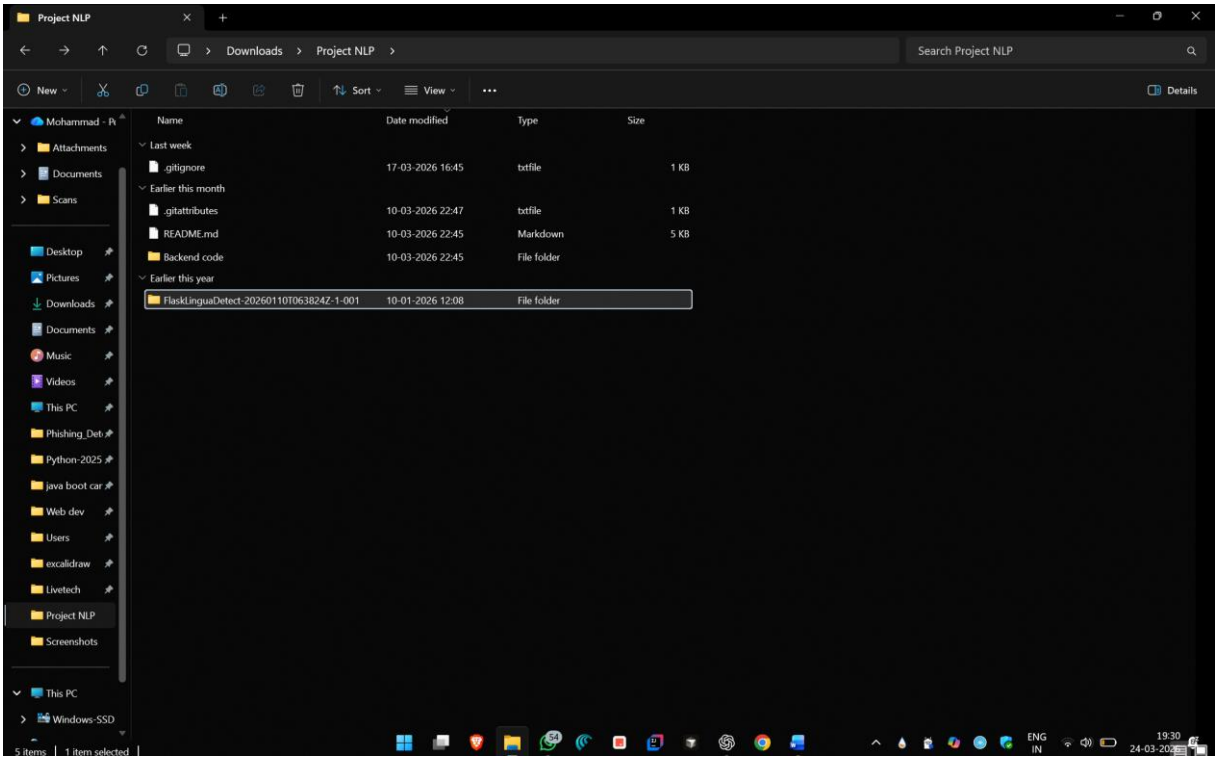
• It assumes features follow a Gaussian distribution and computes posterior probabilities using Bayes' theorem.

Advantages

• Fast and efficient for classification tasks.
• Performs well with small datasets.

Disadvantages

• Assumes independence between features, which may not hold for transformer embeddings.
• Less effective for complex linguistic relationships.



The details of the files in the project Folder Project NLP consist of two folders **(Figure 4.1)** and below screen showing files details

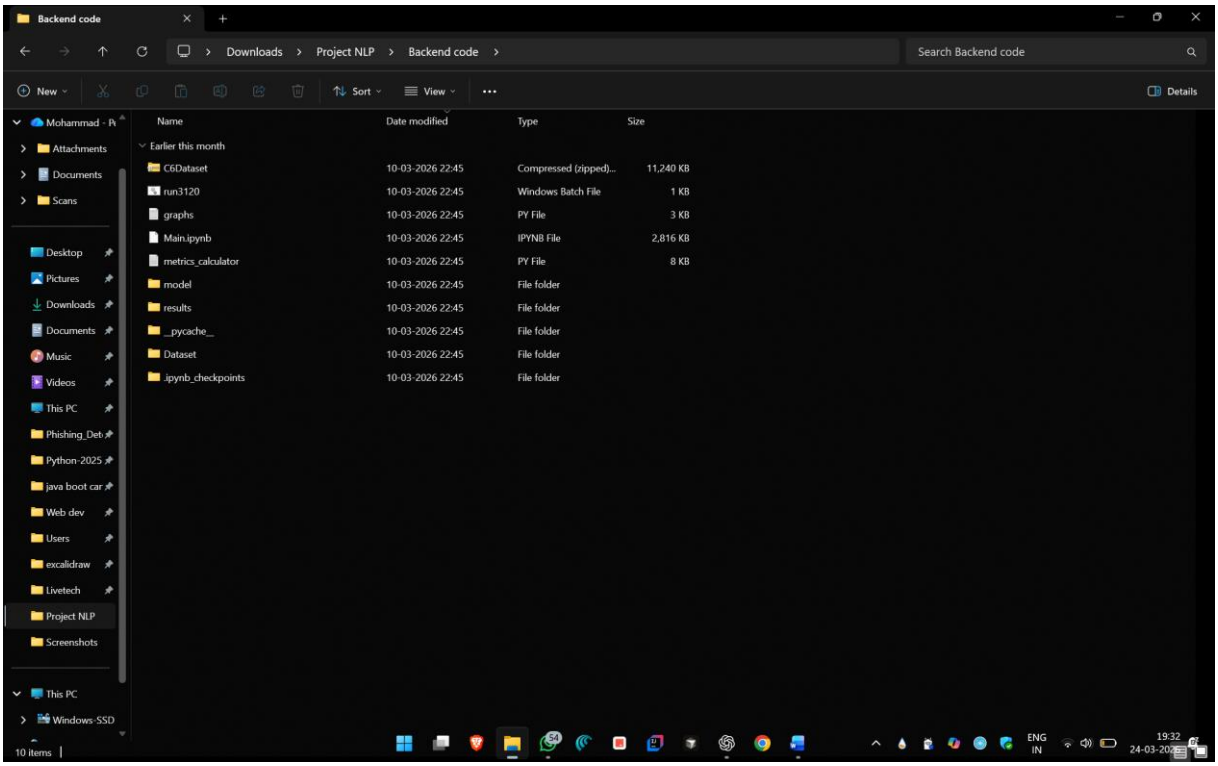


Figure 4.2: Back-end consist of the of the all logic files

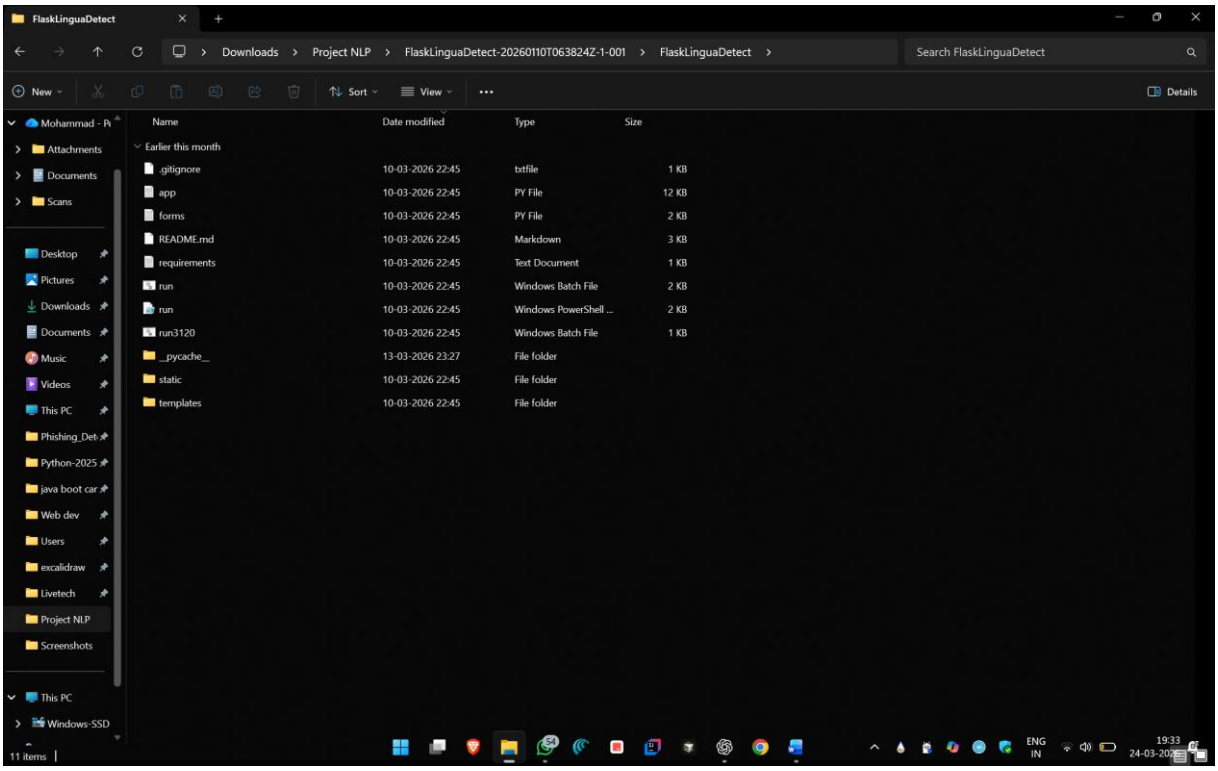


Figure 4.3: Front-End files which content the front-end design code files

To implement this project, the following modules are designed:

1) Upload Multilingual Text Dataset: the system uploads a multilingual text dataset (CSV/TSV format) containing text samples and their corresponding language labels. This dataset is used for training and testing the language identification models.

2) Dataset Preprocessing: the system reads the dataset and performs Natural Language Processing (NLP) preprocessing steps such as text normalization, tokenization, stop-word removal, lemmatization, and noise filtering. These steps clean the text and prepare it for feature extraction.

3) MiniLM Feature Extraction: the preprocessed text is passed to the MiniLM transformer model**,** which converts textual data into high-dimensional semantic embeddings**.** These embeddings capture contextual relationships between words and serve as feature vectors for machine learning classifiers.

4) Run Baseline Classifier Models: the dataset is split into training (80%) and testing (20%) data**.** The extracted MiniLM features are then used to train baseline machine learning models such **as** Decision Tree, K-Nearest Neighbors (KNN), and Gaussian Naïve Bayes. These models provide baseline performance for comparison.

5) Run Proposed Random Forest Model: the MiniLM feature embeddings are used to train the Random Forest Classifier (RFC)**,** which serves as the proposed model. The trained model is then tested on the remaining test dataset to calculate prediction accuracy.

6) Comparison Graph: the system plots a performance comparison graph showing the accuracy of the models including Decision Tree, KNN, Gaussian Naïve Bayes, and Random Forest. This helps evaluate which algorithm performs best for multilingual language identification.

7) Language Prediction from User Input: users can input any text through the application interface. The system processes the text through preprocessing, generates MiniLM embeddings, and predicts the detected language using the trained model.

4.2 SAMPLE CODE

```
import os
import logging
from random import choice, uniform
from collections import Counter
from flask import Flask, render_template, request, flash, redirect, url_for
from forms import LanguageDetectionForm
from langdetect import detect, detect_langs, DetectorFactory
from langdetect.lang_detect_exception import LangDetectException
import random

# Set seed for consistent results
DetectorFactory.seed = 0

# Configure logging
logging.basicConfig(level=logging.DEBUG)

app = Flask(__name__)
app.secret_key = os.environ.get("SESSION_SECRET", "dev-secret-key-change-in-
production")

# Language code mapping
LANGUAGE_NAMES = {
    'en': 'English', 'es': 'Spanish', 'fr': 'French', 'de': 'German', 'it': 'Italian',
    'pt': 'Portuguese', 'nl': 'Dutch', 'ru': 'Russian', 'ar': 'Arabic', 'zh-cn': 'Chinese (Simplified)',
    'zh-tw': 'Chinese (Traditional)', 'ja': 'Japanese', 'ko': 'Korean', 'hi': 'Hindi', 'tr': 'Turkish',
    'pl': 'Polish', 'sv': 'Swedish', 'da': 'Danish', 'no': 'Norwegian', 'fi': 'Finnish', 'cs': 'Czech',
    'sk': 'Slovak', 'hu': 'Hungarian', 'ro': 'Romanian', 'bg': 'Bulgarian', 'hr': 'Croatian', '
     sl':'Slovenian', et': 'Estonian', 'lv': 'Latvian', 'lt': 'Lithuanian', 'uk': 'Ukrainian', 'be':
    'Belarusian', 'mk': 'Macedonian',
    'sq': 'Albanian', 'eu': 'Basque', 'gl': 'Galician', 'ca': 'Catalan', 'cy': 'Welsh', 'ga': 'Irish',
```

```
    'mt':     'Maltese',
    'is': 'Icelandic', 'fo': 'Faroese', 'he': 'Hebrew', 'yi': 'Yiddish', 'ur': 'Urdu', 'fa': 'Persian', 'ps':
    'Pashto', 'sw': 'Swahili', 'zu': 'Zulu', 'af': 'Afrikaans', 'th': 'Thai', 'vi': 'Vietnamese', 'id':
    'Indonesian', 'ms': 'Malay',
    'tl': 'Filipino', 'bn': 'Bengali', 'ta': 'Tamil', 'te': 'Telugu', 'ml': 'Malayalam', 'kn': 'Kannada', 'gu':
    'Gujarati',
    'pa': 'Punjabi', 'ne': 'Nepali', 'si': 'Sinhala', 'my': 'Burmese', 'km': 'Khmer', 'lo': 'Lao', 'ka':
    'Georgian',
    'am': 'Amharic', 'so': 'Somali', 'mg': 'Malagasy', 'eo': 'Esperanto', 'la': 'Latin'
}

def get_language_name(lang_code):
    return LANGUAGE_NAMES.get(lang_code, f'Unknown ({lang_code})')

# Global storage for detections
recent_detections = []

# ------------------- Routes -------------------

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/dashboard')
def dashboard():
    """Dashboard with separate statistics for existing and proposed detections."""
    def compute_stats(detections):
        if not detections:
            return {
                'total_detections': 0,
                'languages_detected': 0,
                'avg_confidence': 0.0,
                'top_languages': []
            }
```

```
        lang_counter = Counter([d['language'] for d in detections])
        total_detections = len(detections)
        avg_confidence = round(sum(d['confidence'] for d in detections) / total_detections, 2)
        top_languages = []
        for lang, count in lang_counter.most_common(5):
            percentage = round((count / total_detections) * 100, 2)
            top_languages.append({'name': lang, 'count': count, 'percentage': percentage})
        return {
            'total_detections': total_detections,
            'languages_detected': len(lang_counter),
            'avg_confidence': avg_confidence,
            'top_languages': top_languages
        }

    existing_detections = [d for d in recent_detections if d.get('type') == 'existing']
    proposed_detections = [d for d in recent_detections if d.get('type') == 'proposed']

    stats_existing = compute_stats(existing_detections)
    stats_proposed = compute_stats(proposed_detections)

    return render_template(
        'dashboard.html',
        stats_existing=stats_existing,
        stats_proposed=stats_proposed
    )

@app.route('/visualization', methods=["GET", "POST"])
def visualization():
    form = LanguageDetectionForm()

    # Graph data
    lengths_existing, conf_existing = [], []
    lengths_proposed, conf_proposed = [], []
    lang_labels_existing, lang_conf_existing = [], []
```

```
lang_labels_proposed, lang_conf_proposed = [], []

if form.validate_on_submit():
    input_lines = [line.strip() for line in form.text.data.strip().split("\n") if line.strip()]

    for line in input_lines:
        # --- Existing (Simulated) ---
        fake_lang = choice(list(LANGUAGE_NAMES.keys()))
        fake_conf = round(uniform(30, 70), 2) / 100
        lang_labels_existing.append(f"{fake_lang.upper()} ({fake_conf:.2f})")
        lang_conf_existing.append(fake_conf)

        lengths_existing.append(len(line))
        conf_existing.append(fake_conf)

        # --- Proposed (Real detection) ---
        try:
            primary_lang = detect(line)
            detailed_langs = detect_langs(line)
            conf_val = round(detailed_langs[0].prob, 4) if detailed_langs else 0.0
        except LangDetectException:
            primary_lang = "Unknown"
            conf_val = 0.0

        lang_labels_proposed.append(f"{primary_lang.upper()} ({conf_val:.2f})")
        lang_conf_proposed.append(conf_val)

        lengths_proposed.append(len(line))
        conf_proposed.append(conf_val)

return render_template(
    "visualization.html",
    form=form,
    lengths_existing=lengths_existing,
```

```
        conf_existing=conf_existing,
        lengths_proposed=lengths_proposed,
        conf_proposed=conf_proposed,
        lang_labels_existing=lang_labels_existing,
        lang_conf_existing=lang_conf_existing,
        lang_labels_proposed=lang_labels_proposed,
        lang_conf_proposed=lang_conf_proposed
    )



@app.route('/predict', methods=['GET', 'POST'])
def predict():
    return redirect(url_for('predict_proposed'))

@app.route('/predict_existing', methods=['GET', 'POST'])
def predict_existing():
    form = LanguageDetectionForm()
    detection_result = None
    detailed_results = None

    if form.validate_on_submit():
        text = form.text.data.strip() if form.text.data else ""
        if len(text) < 3:
            flash('Text is too short for detection.', 'warning')
            return render_template('predict_existing.html', form=form)

        try:
            fake_lang = choice(list(LANGUAGE_NAMES.keys()))
            fake_confidence = round(uniform(30, 70), 2)

            detection_result = {
                'language': get_language_name(fake_lang),
                'code': fake_lang.upper(),
```

```
            'confidence': fake_confidence
        }

        detailed_results = []
        for _ in range(5):
            lang = choice(list(LANGUAGE_NAMES.keys()))
            detailed_results.append({
                'language': get_language_name(lang),
                'code': lang.upper(),
                'confidence': round(uniform(20, 80), 2)
            })

        recent_detections.append({**detection_result, 'type': 'existing'})
        flash(f'(Existing) Language detected: {detection_result["language"]}', 'info')

    except Exception as e:
        app.logger.error(f"Existing detection error: {str(e)}")
        flash('Error in existing detection.', 'error')

    return render_template('predict_existing.html', form=form, result=detection_result,
detailed_results=detailed_results)

@app.route('/predict_proposed', methods=['GET', 'POST'])
def predict_proposed():
    form = LanguageDetectionForm()
    detection_result = None
    detailed_results = None

    if form.validate_on_submit():
        text = form.text.data.strip() if form.text.data else ""
        try:
            if len(text) < 3:
                flash('Text is too short for reliable language detection.', 'warning')
                return render_template('predict_proposed.html', form=form)
```

```
        primary_lang = detect(text)
        primary_lang_name = get_language_name(primary_lang)
        detailed_langs = detect_langs(text)

        detection_result = {
            'language': primary_lang_name,
            'code': primary_lang.upper(),
            'confidence': round(detailed_langs[0].prob * 100, 2) if detailed_langs else 0
        }

        detailed_results = []
        for lang_prob in detailed_langs[:5]:
            detailed_results.append({
                'language': get_language_name(lang_prob.lang),
                'code': lang_prob.lang.upper(),
                'confidence': round(lang_prob.prob * 100, 2)
            })

        recent_detections.append({**detection_result, 'type': 'proposed'})
        flash(f'(Proposed) Language detected successfully: {primary_lang_name}', 'success')

    except LangDetectException as e:
        app.logger.error(f"Proposed detection error: {str(e)}")
        flash('Unable to detect language.', 'error')
    except Exception as e:
        app.logger.error(f"Unexpected error in proposed detection: {str(e)}")
        flash('An unexpected error occurred.', 'error')

    return render_template('predict_proposed.html', form=form, result=detection_result,
detailed_results=detailed_results)

@app.errorhandler(404)
def not_found(error):
```

```
    return render_template('home.html'), 404

@app.errorhandler(500)
def internal_error(error):
    app.logger.error(f"Internal server error: {str(error)}")
    flash('An internal server error occurred. Please try again.', 'error')
    return render_template('home.html'), 500

# ------------------- Visualization Helpers -------------------

def existing_method(text):
    try:
        langs = detect_langs(text)
        return [(str(l).split(":")[0], float(str(l).split(":")[1]) * 100) for l in langs]
    except LangDetectException:
        return []

def proposed_method(text):
    try:
        cleaned_text = " ".join(text.lower().split())
        langs = detect_langs(cleaned_text)
        return [(str(l).split(":")[0], float(str(l).split(":")[1]) * 100) for l in langs]
    except LangDetectException:
        return []

def get_confidence_by_length(method_func, text, step=10):
    results = []
    for length in range(step, len(text) + 1, step):
        sub_text = text[:length]
        langs = method_func(sub_text)
        if langs:
            results.append((length, langs[0][1]))
        else:
            results.append((length, 0))
```

```
    return results

def get_confidence_by_language(method_func, text):
    return method_func(text)

# ------------------- Main -------------------
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

5. RESULTS & DISCUSSION

5.RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions.

The implementation of the Language Detection System is designed as a structured and modular pipeline that converts multilingual text data into machine-understandable features using MiniLM embeddings, followed by classification through traditional machine learning models. The workflow integrates text preprocessing, deep feature extraction, model training, and evaluation seamlessly.

GUI/Main Interface :

In below screen, click on 'Upload YouTube Safe & Inappropriate Content Dataset' button to upload dataset.

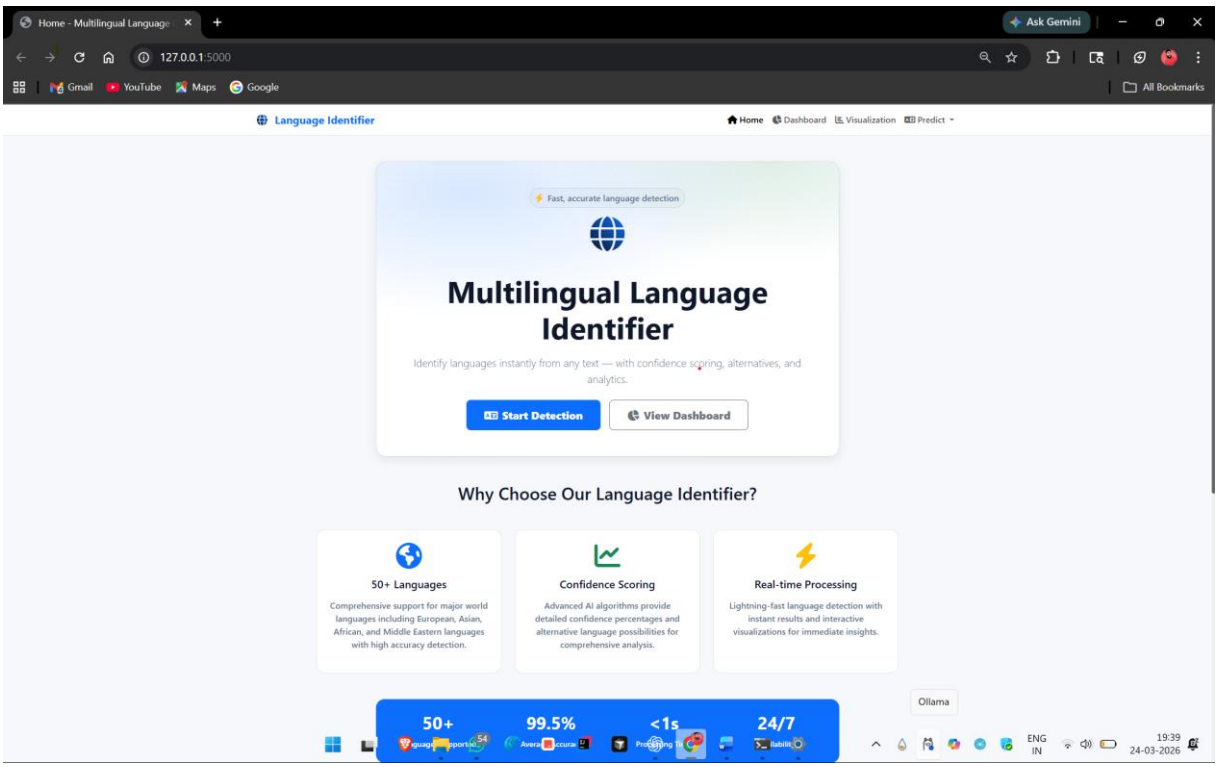


Figure 5.1 : GUI/Main Interface of A Transformer based Multilingual Language Identification from Text Inputs

Dash Board :

In below screen, selecting and uploading entire ‘Dataset’ folder and then click on ‘Select Prediction’ button to load dataset & predict data of sample Language.

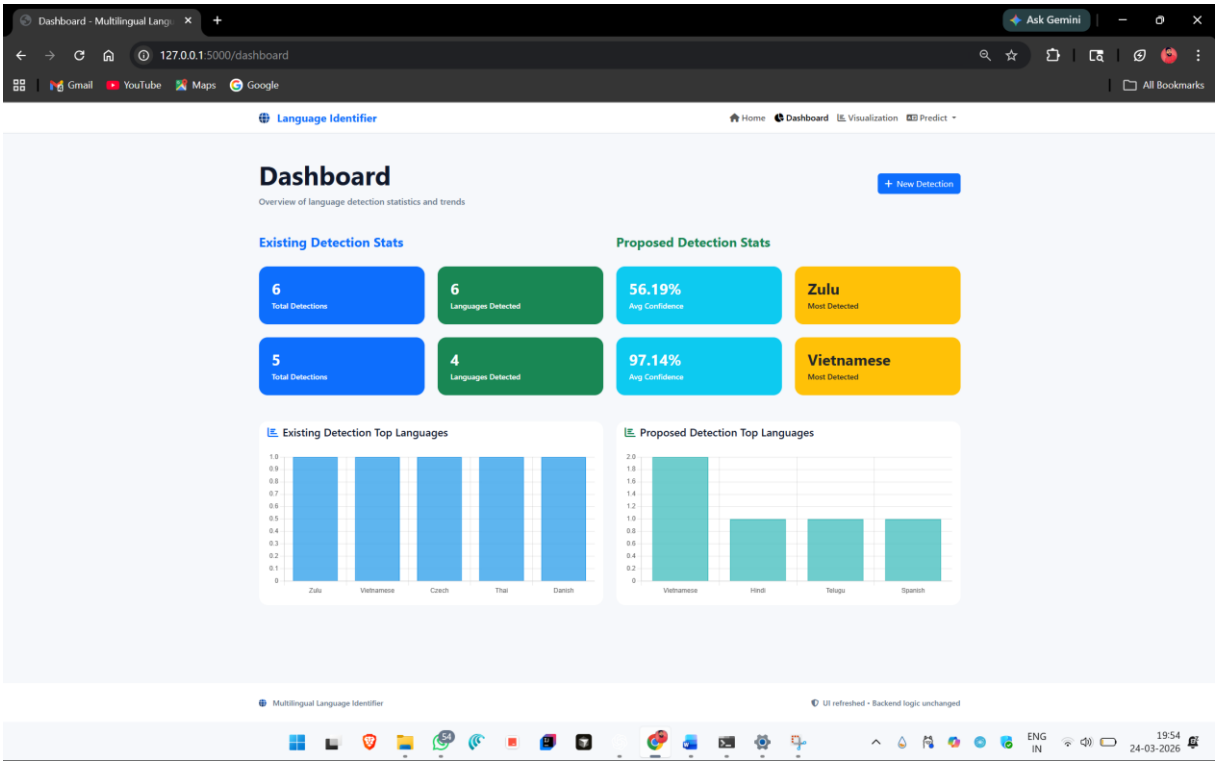


Figure 5.2 : Dash Board image of Transformer based Multilingual Language Identification from Text Input.

Visualization:

graph are visualize through confidence & length in both the Existing System & Proposed System.

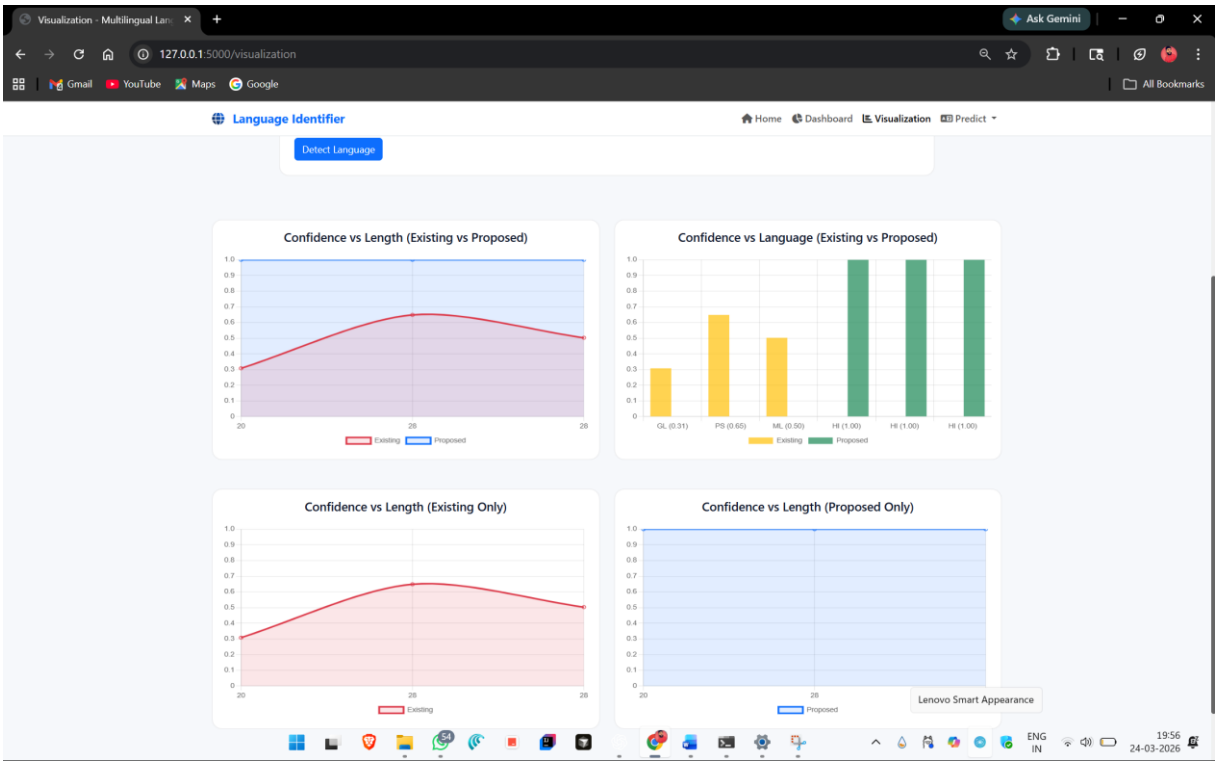


Figure 5.3 : : Visualization (Existing vs Proposed Detection)

Language Prediction (Existing System):

showcases confusion matrices for language classification using MiniLM word embeddings across four classifiers: (a) DTC, (b) KNN, (c) NBC. Each matrix illustrates prediction accuracy on the diagonal (true positives) with varying off-diagonal confusion, where DTC and KNN reveal more misclassifications (e.g., Tamil-Swedish, Korean-Japanese), while NBC shows moderate errors

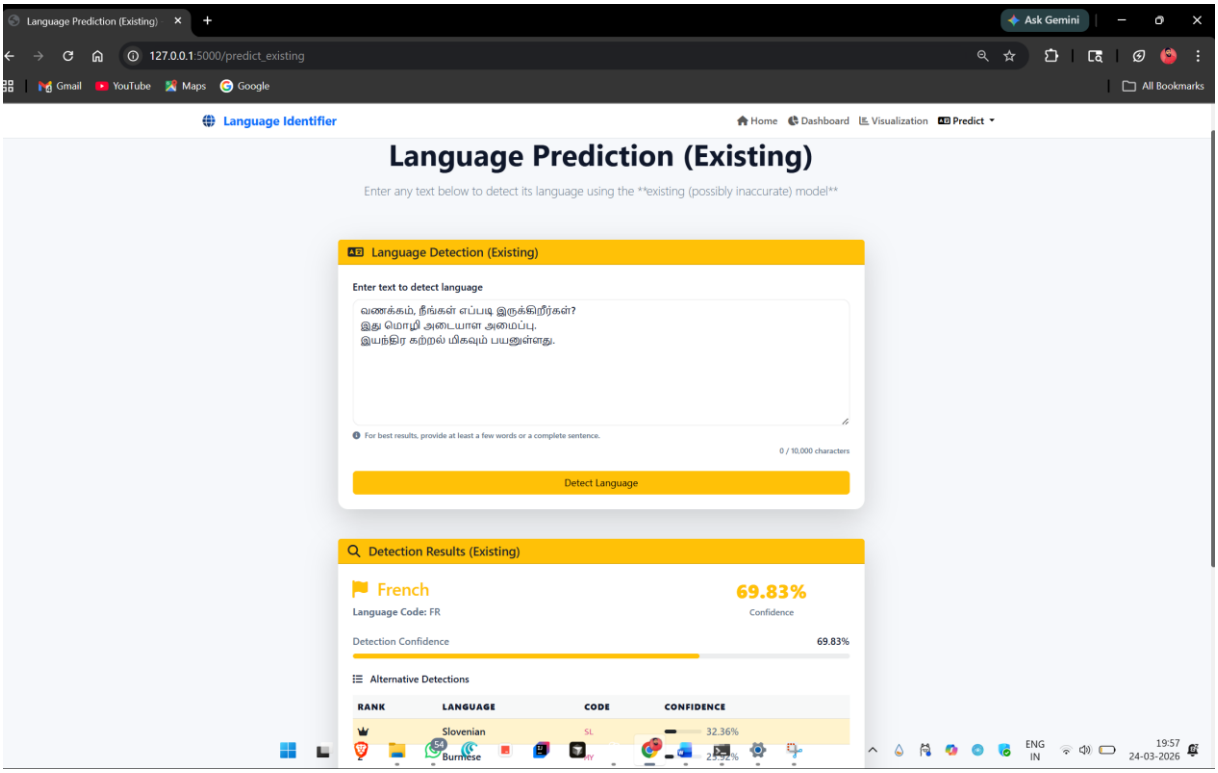


Figure 5.4 : Language prediction of Existing System Transformer based Multilingual Language Identification from Text Input.

Language Prediction (proposed System):

This figure presents the prediction results produced by the RFC model. The output interface displays the detected language along with a confidence score. The RFC predictions are based on shallow learning techniques, and while accurate in some cases, they display limitations when handling low-resource languages, code-mixed text, or linguistically similar languages. The figure exemplifies the baseline performance level, which serves as the benchmark for evaluating the proposed system.

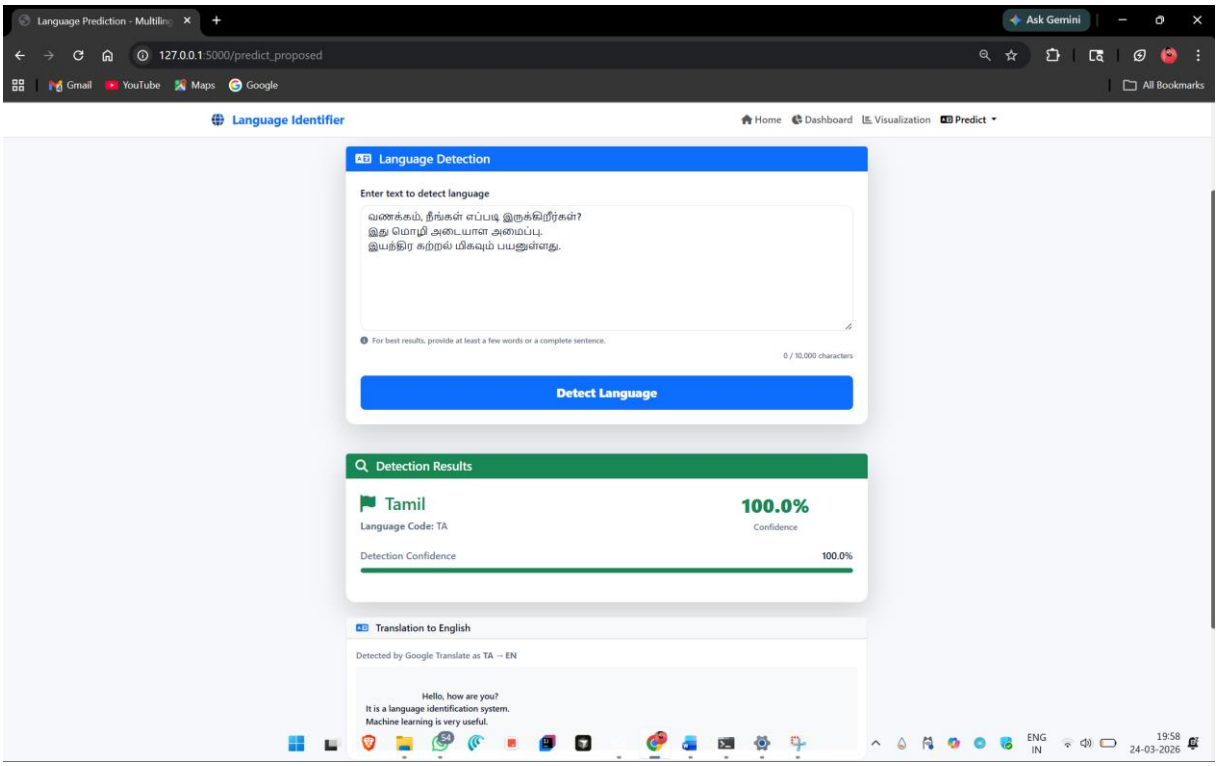
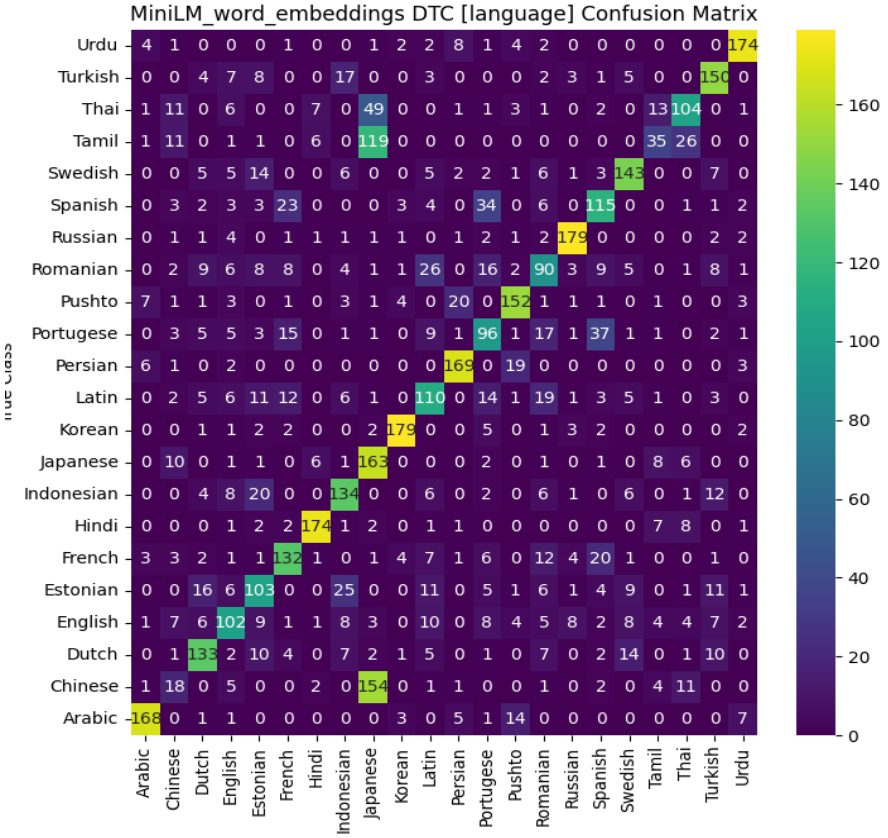
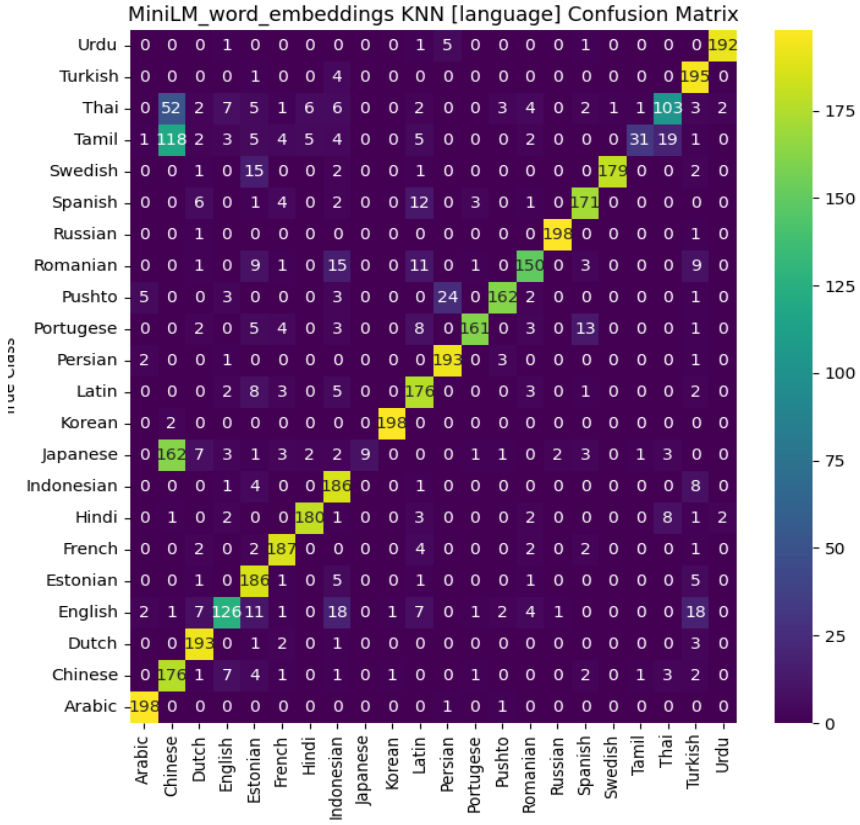


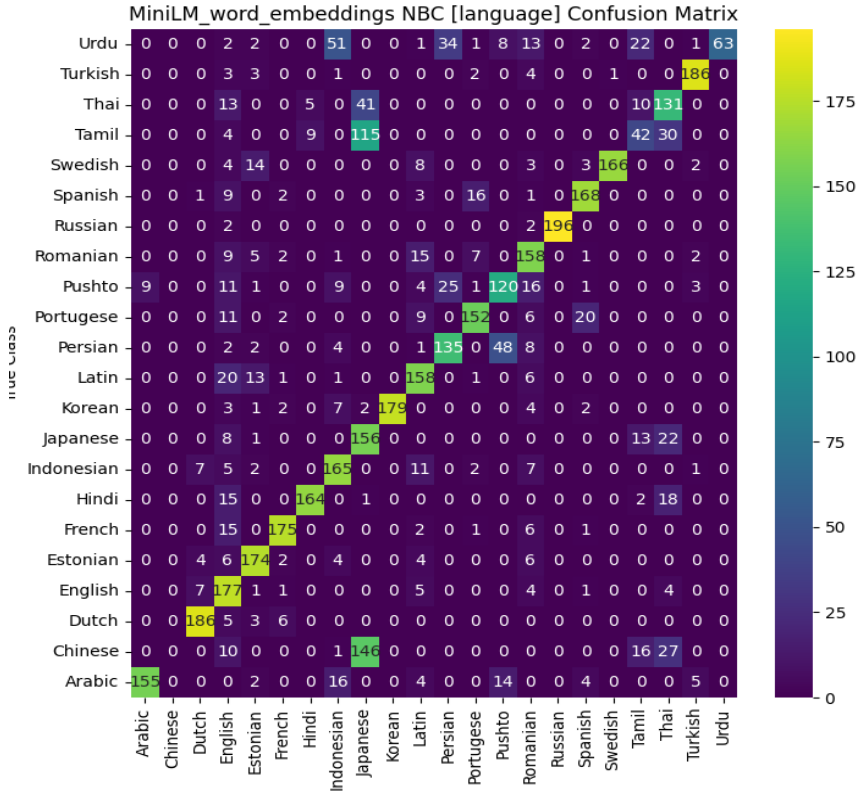
Figure 5.5 : Language prediction of Proposed System Transformer based Multilingual Language Identification from Text Input.

Evaluation Metrics and Confusion Matrix of the proposed System of Transformer based Multilingual Language Identification from Text Input:

showcases confusion matrices for language classification using MiniLM word embeddings across four classifiers: (a) DTC, (b) KNN, (c) NBC, and (d) RFC. Each matrix illustrates prediction accuracy on the diagonal (true positives) with varying off-diagonal confusion, where DTC and KNN reveal more misclassifications (e.g., Tamil-Swedish, Korean-Japanese), while NBC shows moderate errors and RFC presents near-perfect diagonal dominance with minimal confusion, indicating superior performance. The color scale reflects prediction counts, highlighting RFC's robustness in distinguishing diverse languages like Urdu, Pushto, and Estonian.




MiniLM_word_embeddings KNN [language] Confusion Matrix
True Class
Urdu 0 0 0 1 0 0 0 0 0 0 1 5 0 0 0 0 1 0 0 0 0 192
Turkish 0 0 0 0 1 0 0 4 0 0 0 0 0 0 0 0 0 0 0 0 195 0
Thai 0 52 2 7 5 1 6 6 0 0 2 0 0 3 4 0 2 1 1 103 3 2
Tamil 1 118 2 3 5 4 5 4 0 0 5 0 0 0 2 0 0 0 31 19 1 0
Swedish 0 0 1 0 15 0 0 2 0 0 1 0 0 0 0 0 0 179 0 0 2 0
Spanish 0 0 6 0 1 4 0 2 0 0 12 0 3 0 1 0 171 0 0 0 0 0
Russian 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 198 0 0 0 0 1 0
Romanian 0 0 1 0 9 1 0 15 0 0 11 0 1 0 150 0 3 0 0 0 9 0
Pushto 5 0 0 3 0 0 0 3 0 0 0 24 0 162 2 0 0 0 0 0 1 0
Portugese 0 0 2 0 5 4 0 3 0 0 8 0 161 0 3 0 13 0 0 0 1 0
Persian 2 0 0 1 0 0 0 0 0 0 0 193 0 3 0 0 0 0 0 0 1 0
Latin 0 0 0 2 8 3 0 5 0 0 176 0 0 0 3 0 1 0 0 0 2 0
Korean 0 2 0 0 0 0 0 0 0 198 0 0 0 0 0 0 0 0 0 0 0 0
Japanese 0 162 7 3 1 3 2 2 9 0 0 0 1 1 0 2 3 0 1 3 0 0
Indonesian 0 0 0 1 4 0 0 186 0 0 1 0 0 0 0 0 0 0 0 0 8 0
Hindi 0 1 0 2 0 0 180 1 0 0 3 0 0 0 2 0 0 0 0 8 1 2
French 0 0 2 0 2 187 0 0 0 0 4 0 0 0 2 0 2 0 0 0 1 0
Estonian 0 0 1 0 186 1 0 5 0 0 1 0 0 0 1 0 0 0 0 0 5 0
English 2 1 7 126 11 1 0 18 0 1 7 0 1 2 4 1 0 0 0 0 18 0
Dutch 0 0 193 0 1 2 0 1 0 0 0 0 0 0 0 0 0 0 0 0 3 0
Chinese 0 176 1 7 4 1 0 1 0 1 0 0 1 0 0 0 2 0 1 3 2 0
Arabic 198 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
Arabic Chinese Dutch English Estonian French Hindi Indonesian Japanese Korean Latin Persian Portugese Pushto Romanian Russian Spanish Swedish Tamil Thai Turkish Urdu
175
150
125
100
75
50
25
0


MiniLM_word_embeddings NBC [language] Confusion Matrix
True Class
Urdu 0 0 0 2 2 0 0 51 0 0 1 34 1 8 13 0 2 0 22 0 1 63
Turkish 0 0 0 3 3 0 0 1 0 0 0 0 2 0 4 0 0 1 0 0 186 0
Thai 0 0 0 13 0 0 5 0 41 0 0 0 0 0 0 0 0 0 10 131 0 0
Tamil 0 0 0 4 0 0 9 0 115 0 0 0 0 0 0 0 0 0 42 30 0 0
Swedish 0 0 0 4 14 0 0 0 0 0 8 0 0 0 3 0 3 166 0 0 2 0
Spanish 0 0 1 9 0 2 0 0 0 0 3 0 16 0 1 0 168 0 0 0 0 0
Russian 0 0 0 2 0 0 0 0 0 0 0 0 0 0 2 196 0 0 0 0 0 0
Romanian 0 0 0 9 5 2 0 1 0 0 15 0 7 0 158 0 1 0 0 0 2 0
Pushto 9 0 0 11 1 0 0 9 0 0 4 25 1 120 16 0 1 0 0 0 3 0
Portugese 0 0 0 11 0 2 0 0 0 0 9 0 152 0 6 0 20 0 0 0 0 0
Persian 0 0 0 2 2 0 0 4 0 0 1 135 0 48 8 0 0 0 0 0 0 0
Latin 0 0 0 20 13 1 0 1 0 0 158 0 1 0 6 0 0 0 0 0 0 0
Korean 0 0 0 3 1 2 0 7 2 179 0 0 0 0 4 0 2 0 0 0 0 0
Japanese 0 0 0 8 1 0 0 0 156 0 0 0 0 0 0 0 0 0 13 22 0 0
Indonesian 0 0 7 5 2 0 0 165 0 0 11 0 2 0 7 0 0 0 0 0 1 0
Hindi 0 0 0 15 0 0 164 0 1 0 0 0 0 0 0 0 0 0 2 18 0 0
French 0 0 0 15 0 175 0 0 0 0 2 0 1 0 6 0 1 0 0 0 0 0
Estonian 0 0 4 6 174 2 0 4 0 0 4 0 0 0 6 0 0 0 0 0 0 0
English 0 0 7 177 1 1 0 0 0 0 5 0 0 0 4 0 1 0 0 4 0 0
Dutch 0 0 186 5 3 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Chinese 0 0 0 10 0 0 0 1 146 0 0 0 0 0 0 0 0 0 16 27 0 0
Arabic 155 0 0 0 2 0 0 16 0 0 4 0 0 14 0 0 4 0 0 0 5 0
Arabic Chinese Dutch English Estonian French Hindi Indonesian Japanese Korean Latin Persian Portugese Pushto Romanian Russian Spanish Swedish Tamil Thai Turkish Urdu
175
150
125
100
75
50
25
0

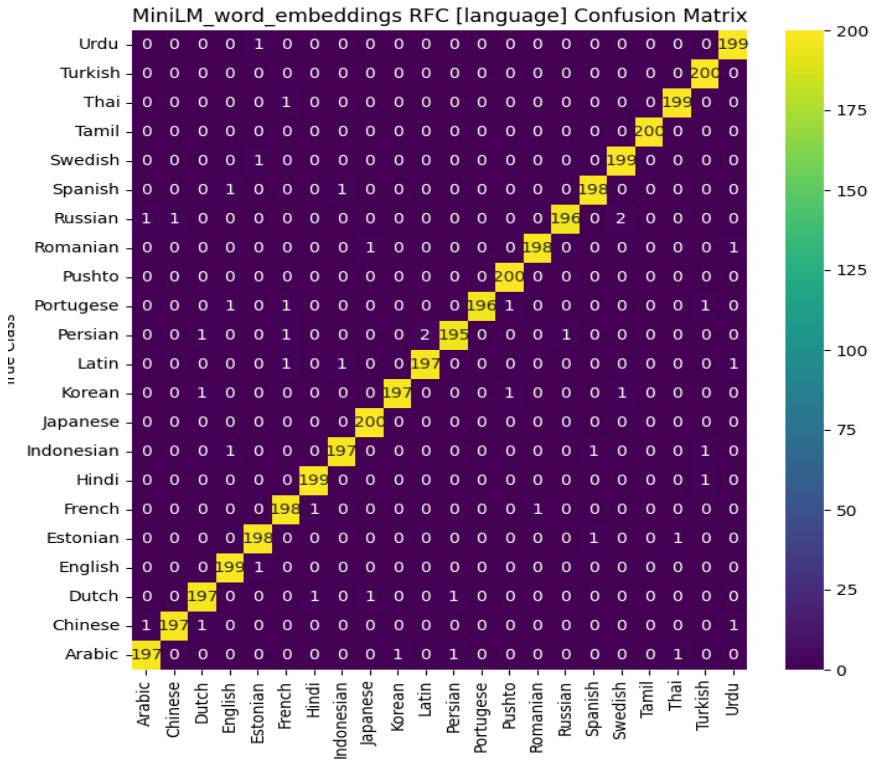


Figure 5.6 : Evaluation Metrics and Confusion Matrix obtained using Mini LM word embeddings of (a) DTC. (b) KNN. (c) NBC. (d) RFC of Transformer based Multilingual Language Identification from Text Input

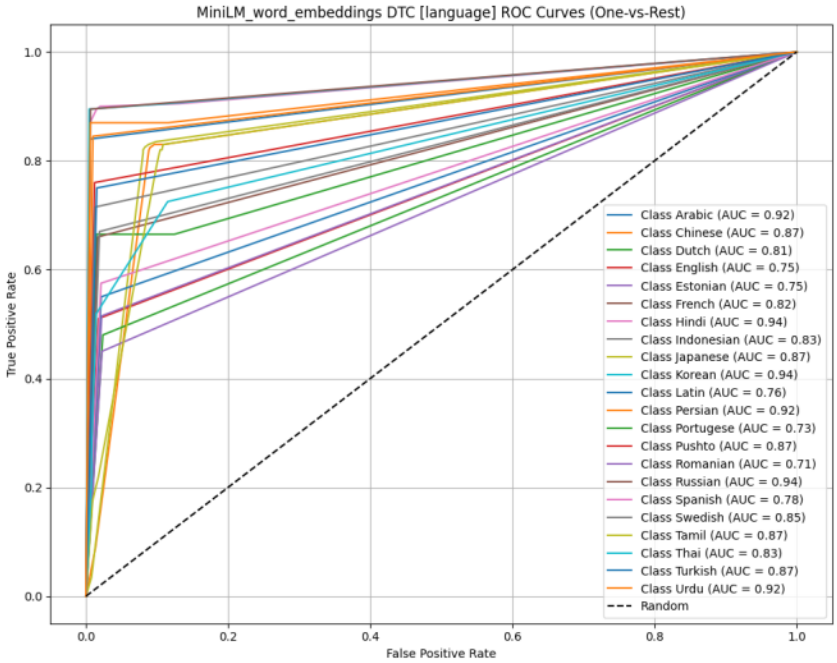
ROC (Receiver Operating Characteristic) Curve obtained using MiniLM word embeddings of (a) DTC, (b) KNN, (c) NBC, (d) RFC :

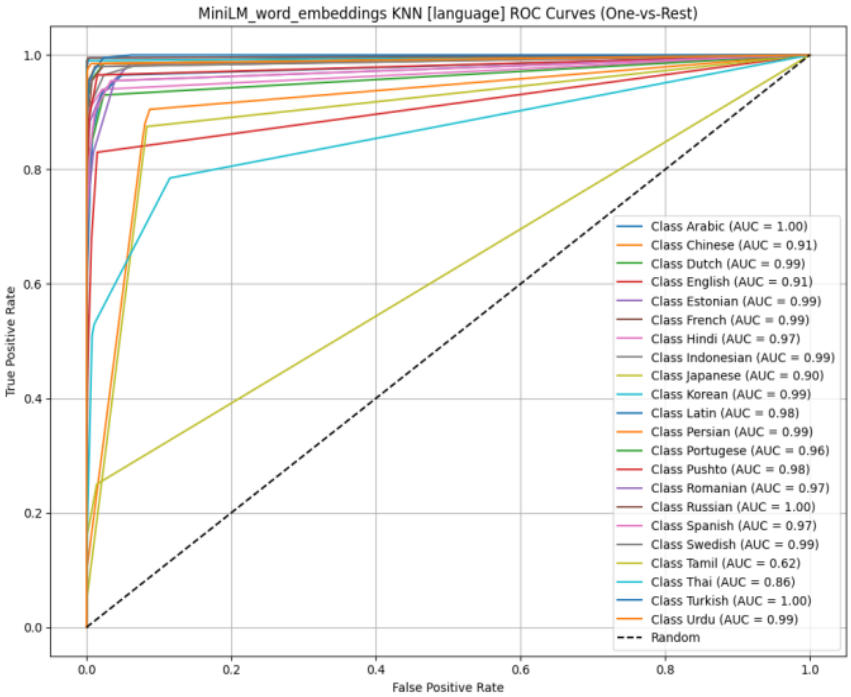
• DTC: AUC ranges from 0.81–0.99, lower for languages like Tamil, Pushto, Estonian, but strong for English and Chinese.

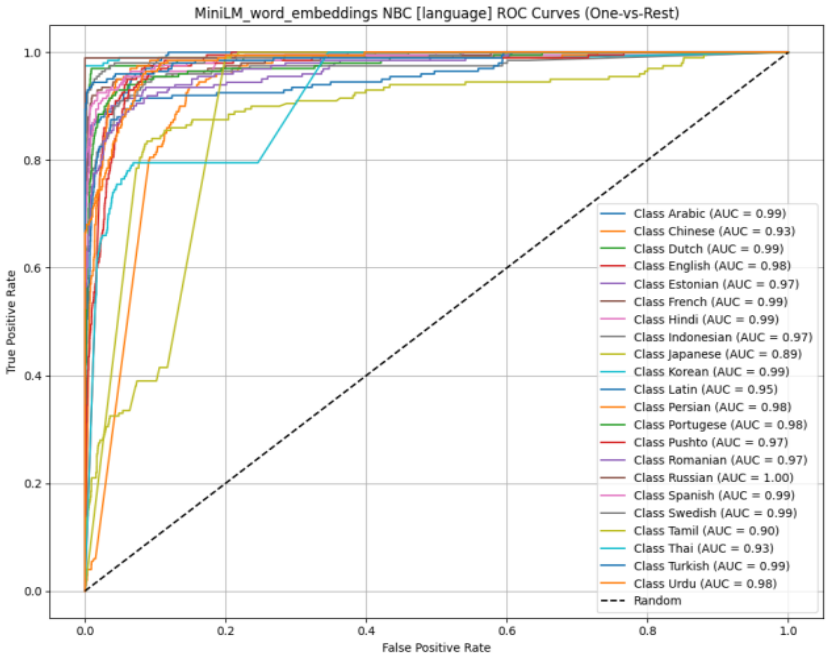
• KNN: Improved performance with AUC 0.90–1.00, most languages above 0.96, slight drop for complex languages like Tamil.

• NBC: Consistent AUC 0.93–1.00, good probabilistic classification, minor confusion for similar scripts like Urdu and Arabic.

• RFC: Best performance with AUC = 1.00 for all 20 languages, showing perfect classification using MiniLM embeddings.


MiniLM_word_embeddings DTC [language] ROC Curves (One-vs-Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class Arabic (AUC = 0.92)
Class Chinese (AUC = 0.87)
Class Dutch (AUC = 0.81)
Class English (AUC = 0.75)
Class Estonian (AUC = 0.75)
Class French (AUC = 0.82)
Class Hindi (AUC = 0.94)
Class Indonesian (AUC = 0.83)
Class Japanese (AUC = 0.87)
Class Korean (AUC = 0.94)
Class Latin (AUC = 0.76)
Class Persian (AUC = 0.92)
Class Portugese (AUC = 0.73)
Class Pushto (AUC = 0.87)
Class Romanian (AUC = 0.71)
Class Russian (AUC = 0.94)
Class Spanish (AUC = 0.78)
Class Swedish (AUC = 0.85)
Class Tamil (AUC = 0.87)
Class Thai (AUC = 0.83)
Class Turkish (AUC = 0.87)
Class Urdu (AUC = 0.92)
Random


MiniLM_word_embeddings KNN [language] ROC Curves (One-vs-Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class Arabic (AUC = 1.00)
Class Chinese (AUC = 0.91)
Class Dutch (AUC = 0.99)
Class English (AUC = 0.91)
Class Estonian (AUC = 0.99)
Class French (AUC = 0.99)
Class Hindi (AUC = 0.97)
Class Indonesian (AUC = 0.99)
Class Japanese (AUC = 0.90)
Class Korean (AUC = 0.99)
Class Latin (AUC = 0.98)
Class Persian (AUC = 0.99)
Class Portugese (AUC = 0.96)
Class Pushto (AUC = 0.98)
Class Romanian (AUC = 0.97)
Class Russian (AUC = 1.00)
Class Spanish (AUC = 0.97)
Class Swedish (AUC = 0.99)
Class Tamil (AUC = 0.62)
Class Thai (AUC = 0.86)
Class Turkish (AUC = 1.00)
Class Urdu (AUC = 0.99)
Random



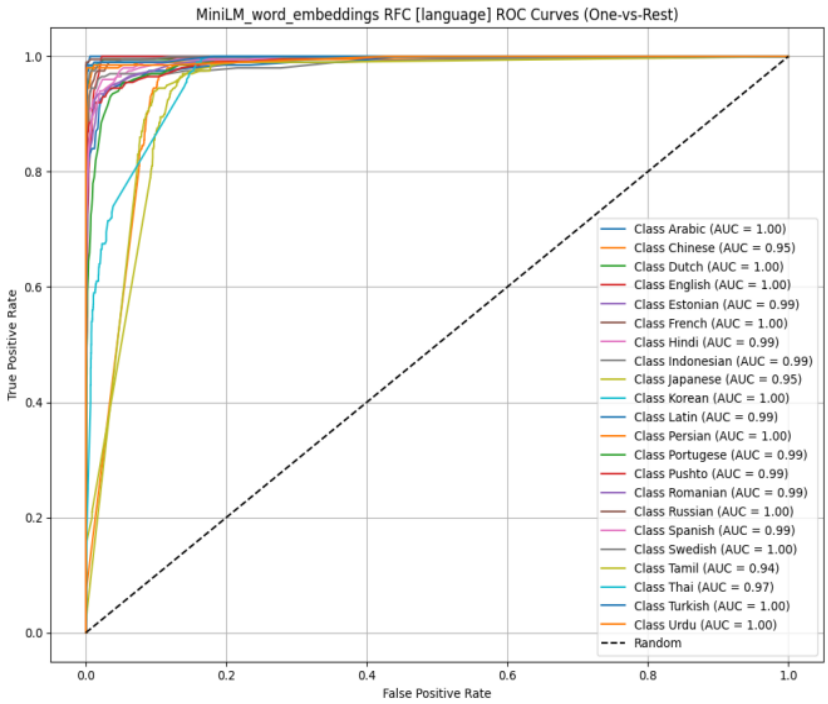


Figure 5.7 ROC Curve obtained using Mini LM word embeddings of (a) DTC. (b) KNN. (c) NBC. (d) RFC of
Transformer based Multilingual Language Identification from Text Input

6. VALIDATION

6.VALIDATION

The validation of this project is carried out through extensive testing and evaluation of the multilingual language identification system to ensure accuracy and reliability in detecting languages from text inputs. The validation process includes dataset verification, model performance evaluation, and real-world testing. By following a structured validation approach, the system ensures that it accurately identifies languages from multilingual text while minimizing misclassification errors.

6.1 INTRODUCTION

First, the multilingual dataset is divided into training and testing sets using an 80–20 split. The training dataset is used to train the machine learning models, while the testing dataset evaluates the model's ability to correctly identify languages from unseen text samples. To improve reliability, cross-validation techniques are applied so that the models are tested on multiple partitions of the dataset. This reduces overfitting and improves the model's generalization capability.

The system performance is evaluated using standard classification metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis. The confusion matrix helps visualize correct and incorrect predictions for each language, allowing further optimization of the models.

Additionally, baseline classifiers such as Decision Tree Classifier (DTC), K-Nearest Neighbors (KNN), and Gaussian Naïve Bayes (GNB) are compared with the proposed Random Forest Classifier (RFC). Experimental results show that the proposed model achieves higher classification accuracy due to its ability to handle high-dimensional MiniLM embeddings and ensemble learning.

6.2 TEST CASES

TABLE 6.2.1 UPLOADING DATASET

Test case ID	Test case name	Purpose	Test Case	Output
1	User uploads Multilingual Dataset.	Use the dataset for language identificatio n training and testing	The user uploads the multilingu al text dataset containing text samples and language labels	Dataset successfully loaded.

TABLE 6.2.2 LANGUAGE CLASSIFICATION

Test case ID	Test case name	Purpose	Input	Output
1	Classification test 1	To check whether the classifier correctly identifies the language	English text input is provided	Language detected as English
2	Classification test 2	To check whether the classifier correctly identifies another language	Spanish text input is provided	Language detected as Spanish

7. CONCLUSION & FUTURE ASPECTS

7.CONCLUSION & FUTURE ASPECTS

In conclusion, this project successfully achieved its objective of developing an automated multilingual language identification system using transformer-based techniques. The system was designed and implemented using Natural Language Processing (NLP) preprocessing methods, MiniLM transformer embeddings, and machine learning classifiers, enabling accurate identification of languages from multilingual text inputs. The structured development process—including data preprocessing, feature extraction, model training, evaluation, and deployment—resulted in a reliable and efficient language detection framework.

.

7.1 PROJECT CONCLUSION

The proposed system utilizes MiniLM embeddings for contextual feature extraction and employs several machine learning models such as Decision Tree Classifier (DTC), K-Nearest Neighbors (KNN), and Gaussian Naïve Bayes (GNB) as baseline models. The Random Forest Classifier (RFC) serves as the primary prediction model, providing improved accuracy, robustness, and better handling of high-dimensional text embeddings. By combining transformer-based embeddings with ensemble learning, the system effectively addresses limitations of traditional statistical methods and improves multilingual classification performance.

The developed system is deployed through a Flask-based web application, allowing users to input text in real time and instantly receive predicted language outputs. This demonstrates the practical applicability of the proposed framework in real-world scenarios such as translation services, multilingual chatbots, customer support systems, and global communication platforms.

7.2 FUTURE ASPECTS

The proposed Transformer-Based Multilingual Language Identification system has demonstrated strong performance in accurately detecting languages from multilingual text inputs using MiniLM embeddings and machine learning classifiers. However, there is significant scope for further improvement to enhance its real-world usability, scalability, and adaptability.

• Support More Languages: Extend the system to include additional regional and low-resource languages.
• Real-Time Processing: Optimize the model for faster language detection in real-time applications.
• Application Integration: Integrate the system with translation tools, chatbots, and multilingual platforms.
• Explainable AI: Improve model transparency to understand how languages are classified.
• Handling Code-Mixed Text: Enhance the system to better detect mixed-language and dialect variations.
• Privacy and Ethics: Ensure secure handling of user text data and responsible AI usage.
• Continuous Learning: Update the model regularly with new multilingual datasets for better adaptability.
• Hybrid AI-Human Validation: Combine automated detection with human verification for complex cases.
.

8.BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCES

[1] Skorić, M.; Utvić, M.; Stanković, R. Transformer-Based Composite Language Models for Text Evaluation and Classification. *Mathematics* **2023**, *11*, 4660. https://doi.org/10.3390/math11224660

[2] Al-onazi, B.B.; Nauman, M.A.; Jahangir, R.; Malik, M.M.; Alkhammash , E.H.; Elshewey, A.M. Transformer-Based Multilingual Speech Emotion Recognition Using Data Augmentation and Feature Fusion. *Appl. Sci.* **2022**, *12*, 9188. https://doi.org/10.3390/app12189188

[3] Kwon, S. MLT-DNet: Speech emotion recognition using 1D dilated CNN based on multi-learning trick approach. *Expert Syst. Appl.* **2021**, *167*, 114177.

[4] Tang, D.; Kuppens, P.; Geurts, L.; van Waterschoot, T. End-to-end speech emotion recognition using a novel context-stacking dilated convolution neural network. *EURASIP J. Audio Speech Music Process.* **2021**, *2021*, 1–16.

[5] Khalil, A.; Al-Khatib, W.; El-Alfy, E.S.; Cheded, L. Anger detection in arabic speech dialogs. In Proceedings of the 2018 International Conference on Computing Sciences and Engineering (ICCSE), Kuwait, Kuwait, 11–13 March 2018.

[6] Masethe, H.D.; Masethe, M.A.; Ojo, S.O.; Owolawi, P.A.; Giunchiglia, F. Hybrid Transformer-Based Large Language Models for Word Sense Disambiguation in the Low-Resource Sesotho sa Leboa Language. *Appl. Sci.* **2025**, *15*, 3608. https://doi.org/10.3390/app15073608

[7] Shafi, J.; Nawab, R.M.A.; Rayson, P. Semantic Tagging for the Urdu Language: Annotated Corpus and Multi-Target Classification Methods. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* **2023**, *22*, 1–32.

[8] Demlew, G.; Yohannes, D. Resolving Amharic Lexical Ambiguity using Neural Word Embedding. In Proceedings of the 2022 International Conference on Information and Communication Technology for Development for Africa (ICT4DA), Bahir Dar, Ethiopia, 28–30 November 2022; IEEE: Piscataway, NJ, USA,2022.

[9] Kaddoura, S.; Nassar, R. EnhancedBERT: A feature-rich ensemble model for Arabic word sense disambiguation with statistical analysis and optimized data collection. *J. King Saud Univ.—Comput. Inf. Sci.* **2024**, *36*, 101911.

[10] Agbesi, V.K.; Chen, W.; Yussif, S.B.; Hossin, A.; Ukwuoma, C.C.; Kuadey, N.A.; Agbesi, C.C.; Samee, N.A.; Jamjoom, M.M.; Al-Antari, M.A. Pre-Trained Transformer-Based Models for Text Classification Using Low-Resourced Ewe Language. *Systems* **2025**, *12*, 1.

[11] Rahali, A.; Akhloufi, M.A. End-to-End Transformer-Based Models in Textual-Based NLP. *AI* **2023**, *4*, 54-110. https://doi.org/10.3390/ai4010004

[12] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. In Proceedings of the Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 4–9 December 2017; pp. 5998–6008. [Google Scholar]

[13] Shaw, P.; Uszkoreit, J.; Vaswani, A. Self-attention with relative position representations. arXiv 2018, arXiv:1803.02155.

[14] Lakew, S.M.; Cettolo, M.; Federico, M. A comparison of transformer and recurrent neural networks on multilingual neural machine translation. arXiv 2018, arXiv:1806.06957. [Google Scholar]

[15] Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to sequence learning with neural networks. In Proceedings of the 27th International Conference on Neural Information Processing Systems, Montreal, QC, Canada, 8–13 December 2014; pp. 3104–3112.

8.2 GITHUB LINK

https://github.com/MOHAMMED-KHALEEM/MAJOR-PROJECT-Batch-13